

Lyapunov-Guided Deep Reinforcement Learning for Stable Online Computation Offloading in Mobile-Edge Computing Networks

Suzhi Bi^{ID}, Senior Member, IEEE, Liang Huang^{ID}, Member, IEEE, Hui Wang,
and Ying-Jun Angela Zhang^{ID}, Fellow, IEEE

Abstract—Opportunistic computation offloading is an effective method to improve the computation performance of mobile-edge computing (MEC) networks under dynamic edge environment. In this paper, we consider a multi-user MEC network with time-varying wireless channels and stochastic user task data arrivals in sequential time frames. In particular, we aim to design an online computation offloading algorithm to maximize the network data processing capability subject to the long-term data queue stability and average power constraints. The online algorithm is practical in the sense that the decisions for each time frame are made without the assumption of knowing the future realizations of random channel conditions and data arrivals. We formulate the problem as a multi-stage stochastic mixed integer non-linear programming (MINLP) problem that jointly determines the binary offloading (each user computes the task either locally or at the edge server) and system resource allocation decisions in sequential time frames. To address the coupling in the decisions of different time frames, we propose a novel framework, named LyDROO, that combines the advantages of Lyapunov optimization and deep reinforcement learning (DRL). Specifically, LyDROO first applies Lyapunov optimization to decouple the multi-stage stochastic MINLP into deterministic per-frame MINLP subproblems. By doing so, it guarantees to satisfy all the long-term constraints by solving the per-frame

subproblems that are much smaller in size. Then, LyDROO integrates model-based optimization and model-free DRL to solve the per-frame MINLP problems with very low computational complexity. Simulation results show that under various network setups, the proposed LyDROO achieves optimal computation performance while stabilizing all queues in the system. Besides, it induces very low computation time that is particularly suitable for real-time implementation in fast fading environments.

Index Terms—Mobile edge computing, resource allocation, Lyapunov optimization, deep reinforcement learning.

I. INTRODUCTION

A. Motivations and Summary of Contributions

THE emerging mobile-edge computing (MEC) technology is widely recognized as a key solution to enhance the computation performance of wireless devices (WDs) [2], especially for size-constrained IoT (Internet of Things) devices with low on-device battery and computing capability. With MEC servers deployed at the edge of radio access networks, e.g., cellular base stations, WDs can offload intensive computation tasks to the edge server (ES) in the vicinity to reduce the computation energy and time cost. Compared to the naive scheme that offloads all the tasks for edge execution, *opportunistic computation offloading*, which dynamically assigns tasks to be computed either locally or at the ES, has shown significant performance improvement under time-varying network conditions, such as wireless channel gains [3], harvested energy level [4], task input-output dependency [5], and edge caching availability [6], etc.

There have been extensive studies on opportunistic computation offloading to optimize the computation performance of multi-user MEC networks [5]–[9]. In general, it involves solving a mixed integer non-linear programming (MINLP) that jointly determines the binary offloading (i.e., either offloading the computation or not) and the communication/computation resource allocation (e.g., task offloading time and local/edge CPU frequencies) decisions. Solving such problems typically requires prohibitively high computational complexity especially in large-size networks. Accordingly, many works have focused on designing reduced-complexity sub-optimal algorithms, such as local-search based heuristics [5], [8], decomposition-oriented search [8], and convex relaxations of the binary variables [9], [25], etc. However, aside from performance losses, the above sub-optimal algorithms still

Manuscript received September 28, 2020; revised February 24, 2021; accepted May 17, 2021. Date of publication June 9, 2021; date of current version November 11, 2021. This work was supported in part by the National Natural Science Foundation of China under Project 61871271 and Project 62072410, in part by the Zhejiang Provincial Natural Science Foundation of China under Project LY19F020033, in part by the General Research Fund established by the Research Grants Council of Hong Kong under Project 14208017 and Project 14201920, in part by the Guangdong Province Pearl River Scholar Funding Scheme 2018, in part by the Key Project of Department of Education of Guangdong Province under Grant 2020ZDZX3050, and in part by the Foundation of Shenzhen City under Project JCYJ20190808120415286. This article will be presented in part at the IEEE International Conference on Communications, Montreal, Canada, June 14–18, 2021 [1]. The associate editor coordinating the review of this article and approving it for publication was D. Li. (Corresponding author: Liang Huang.)

Suzhi Bi is with the College of Electronics and Information Engineering, Shenzhen University, Shenzhen 518060, China, and also with the Peng Cheng Laboratory, Shenzhen 518066, China (e-mail: bsz@szu.edu.cn).

Liang Huang is with the College of Computer Science and Technology, Zhejiang University of Technology, Hangzhou 310023, China (e-mail: lianghuang@zjut.edu.cn).

Hui Wang is with the Shenzhen Institute of Information Technology, Shenzhen 518172, China (e-mail: wanghui@szit.edu.cn).

Ying-Jun Angela Zhang is with the Department of Information Engineering, The Chinese University of Hong Kong, Hong Kong (e-mail: yjzhang@ie.cuhk.edu.hk).

Data is available on-line at <https://github.com/revenol/LyDROO>

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TWC.2021.3085319>.

Digital Object Identifier 10.1109/TWC.2021.3085319

require a large number of numerical iterations to produce a satisfying solution. In practice, the MINLP needs to be frequently re-solved once the system parameters, such as wireless link quality, vary. It is therefore too costly to implement the conventional optimization algorithms in a highly dynamic MEC environment.

The recent development of data-driven *deep reinforcement learning* (DRL) provides a promising alternative to tackle the online computation offloading problem. In a nutshell, the DRL framework takes a model-free approach that uses deep neural networks (DNNs) to directly learn the optimal mapping from the “state” (e.g., time-varying system parameters) to the “action” (e.g., offloading decisions and resource allocation) to maximize the “reward” (e.g., data processing rate) via repeated interactions with the environment [10]. It eliminates the complicated computation of MINLP and automatically learns from the past experience on-the-fly without requiring manually labeled training data samples, and thus is particularly advantageous for online implementation. Many studies have applied DRL techniques to design online offloading algorithms in MEC networks [11]–[20]. In particular, our previous work [18] proposes a hybrid framework, named DROO (Deep Reinforcement learning-based Online Offloading), to combine the advantages of conventional model-based optimization and model-free DRL methods. DROO implements a DNN to produce binary offloading decisions based on the input environment parameters such as channel conditions. The candidate offloading solutions are then fed into a model-based optimization module, which accordingly optimizes the communication/computation resource allocation and outputs an accurate estimate of the reward value for each candidate offloading decision. The integrated learning and optimization approach leads to more robust and faster convergence of the online training process, thanks to the accurate estimation of reward values corresponding to each sampled action.

Apart from optimizing the computation performance, it is equally important to guarantee stable system operation, such as data queue stability and average power consumption. However, most of the existing DRL-based methods do not impose long-term performance constraints (e.g., [11]–[20]). Instead, they resort to heuristic approaches that discourage unfavorable actions in each time frame by introducing penalty terms related to, for example, packet drop events [14], [15] and energy consumption [12], [20]. A well-known framework for online joint utility maximization and stability control is *Lyapunov optimization* [21]. It decouples a multi-stage stochastic optimization to sequential per-stage deterministic subproblems, while providing theoretical guarantee to long-term system stability. Some recent works have applied Lyapunov optimization to design computation offloading strategy in MEC networks (e.g., [22]–[26]). However, it still needs to solve a hard MINLP in each per-stage subproblem to obtain the joint binary offloading and resource allocation decisions. To tackle the intractability, some works have designed reduced-complexity heuristics, such as continuous relaxation in [25] and decoupling heuristic in [26]. This, however, suffers from the similar performance-complexity tradeoff dilemma as in [5], [6], [8], [9].

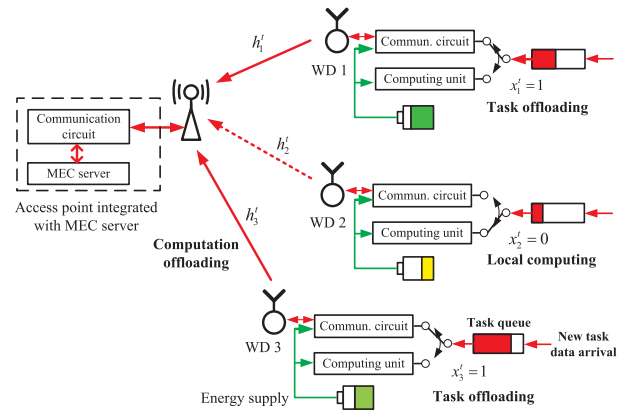


Fig. 1. The considered multi-user MEC network in a tagged time frame.

In this paper, we consider a multi-user MEC network in Fig. 1, where the computation task data arrive at the WDs’ data queues stochastically in sequential time frames. We aim to design an online computation offloading algorithm, in the sense that the decisions for each time frame are made without the assumption of knowing the future realizations of random channel conditions and data arrivals. The objective is to maximize the network data processing capability subject to the long-term data queue stability and average power constraints. To tackle the problem, we propose a Lyapunov-guided Deep Reinforcement learning (DRL)-based Online Offloading (LyDROO) framework that combines the advantages of Lyapunov optimization and DRL. Under fast-varying channel fading and dynamic task arrivals, LyDROO can make online optimal decisions in real time, while guaranteeing the long-term system stability. To the authors’ best knowledge, this is the first work that combines Lyapunov optimization and DRL for online computation offloading design in MEC networks. The main contributions of the paper are:

- *Online stable computation offloading design:* Considering random fading channels and data arrivals, we formulate the problem as a multi-stage stochastic MINLP to maximize the long-term average weighted sum computation rate (i.e., the number of processed bits per second) of all the WDs, subject to the queue stability and average power constraints. In particular, we will make the optimal offloading and resource allocation decisions in each time frame without the assumption of knowing the future realizations of random channel conditions and data arrivals.
- *Integrated Lyapunov-DRL framework:* To tackle the problem, we propose a novel LyDROO framework that combines the advantages of Lyapunov optimization and DRL. In particular, we first apply Lyapunov optimization to decouple the multi-stage stochastic MINLP into per-frame deterministic MINLP problems. Then in each frame, we integrate model-based optimization and model-free DRL to solve the per-frame MINLP problems with very low computational complexity. In particular, we show that the proposed LyDROO framework not only ensures the long-term queue stability and average power

constraints, but also obtains the optimal computation rate performance in an online fashion.

- *Integrated optimization and learning:* LyDROO adopts an actor-critic structure to solve the per-frame MINLP problem. The actor module is a DNN that learns the optimal binary offloading action based on the input environment parameters including the channel gains and queue backlogs of all the WDs. The critic module evaluates the binary offloading action by analytically solving the optimal resource allocation problem. Compared to the conventional actor-critic structure that uses a model-free DNN in the critic module, the proposed approach takes advantage of model information to acquire accurate evaluation of the action, and thus enjoying more robust and faster convergence of the DRL training process.
- *Balanced exploration and exploitation:* LyDROO deploys a noisy order-preserving quantization method to generate offloading action, which elegantly balances the exploration-exploitation tradeoff (i.e., performance or diversity oriented) in the DRL algorithm design to ensure fast training convergence. Besides, the quantization method can adaptively adjust its parameter during the training process, which yields significant reduction in computational complexity without compromising the convergence performance.

Simulation results show that the proposed LyDROO algorithm converges very fast to the optimal computation rate while meeting all the long-term stability constraints. Compared to a myopic benchmark algorithm that greedily maximizes the computation rate in each time frame, the proposed LyDROO achieves a much larger *stable capacity region* that can stabilize the data queues under much heavier task data arrivals and more stringent power constraint.

B. Related Works

Binary and partial computation offloading are two common offloading models in edge computing systems. While the former requires the entire dataset of a computation task to be processed as a whole either locally at a wireless device (WD) or remotely at the edge server, the latter allows the dataset to be partitioned and executed in parallel at both the WD and the edge server [2]. In this paper, we focus on the design of online binary offloading strategy, which is widely adopted in IoT networks for executing simple computation tasks with non-partitionable dataset. Meanwhile, we discuss in Section VII the application of the proposed LyDROO scheme to design online partial offloading strategy when the computation task consists of multiple independent subtasks.

Reduced-complexity algorithms have been widely explored in the literature to tackle the intractability of combinatorial computation offloading problem in multi-user MEC networks adopting binary offloading model. For instance, [7] considers WDs offloading their tasks to the neighboring nodes that arrive and departure in random. It formulates an online stopping problem and proposes a low-complexity algorithm, where each WD individually selects the best set of neighboring nodes

in an online manner to minimize the worst-case computation latency. The proposed method, however, is not suitable for optimizing a long-term average objective considered in this paper. [8] proposes a coordinate descent method that iteratively finds the local-optimum by flipping the binary offloading decision of one user at a time. [5] applies Gibbs sampling to search the decision space in a stochastic manner. To reduce the search dimensions, [8] proposes an ADMM (alternating direction method of multipliers) based method that decomposes the original combinatorial optimization into parallel one-dimension sub-problems. Besides the search-based meta-heuristic algorithms, existing work has also applied convex relaxation to handle the binary variables, such as linear relaxation [8], [25] and quadratic approximation [9]. The aforementioned optimization methods, however, inevitably encounter the performance-complexity tradeoff dilemma when handling integer variables, and are not suitable for online implementation that requires consistently high solution quality under fast-varying environment.

DRL has recently appeared as a promising alternative to solve online computation offloading problems in MEC networks. Existing DRL-based methods take either value-based or policy-based approach to learn the optimal mapping from the “state” (e.g., time-varying system parameters) to the “action” (e.g., offloading decisions and resource allocation). Commonly used value-based DRL methods include deep Q-learning network (DQN) [11]–[13], double DQN [14] and dueling DQN [15], where a DNN is trained to estimate the state-action value function. However, DQN-based methods are costly when the number of possible discrete offloading actions is large, e.g., exponential in the number of WDs. To resolve this issue, recent works have applied policy-based approach, such as the actor-critic DRL [16]–[18] and the deep deterministic policy gradient (DDPG) methods [19], [20], to directly construct the optimal mapping policy from the input state to the output action using a DNN. For example, [19] considers a WD taking only discrete offloading actions, including integer offloading decision and discredited transmit power and offloading rate, and applies an actor-critic DRL method to learn the optimal mapping from continuous input state to the discrete output actions. [18] and [20] train two separate learning modules to generate discrete offloading decision and continuous resource allocation sequentially. Specifically, [20] applies an actor DNN to generate the resource allocation solution, concatenated by a DQN-based critic network to select the discrete offloading action. Similar to [11]–[15], the estimation of state-action value function in the critic network is difficult when the number of possible offloading actions is large. On the other hand, the DROO framework proposed in [18] uses an actor DNN to generate a small number of binary offloading decisions, followed by a model-based critic module that selects the best action by analytically solving the optimal resource allocation problem. Thanks to the accurate evaluation of action acquired by the critic module, DROO enjoys fast convergence to the optimal solution even when the actor DNN provides very few actions (e.g., two actions after sufficient iterations) for the critic to select from. In this paper, we embed DROO in the LyDROO framework to solve the per-frame MINLP problems.

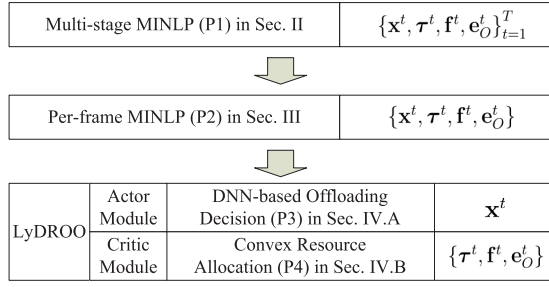


Fig. 2. Organization of the paper.

The above DRL-based methods fail to address the long-term performance requirements, e.g., queue stability and average power, under random environments. In this regard, recent studies have applied Lyapunov optimization to design an online offloading strategy with long-term performance guarantee [22]–[26]. Lyapunov optimization decouples the multi-stage stochastic problem to per-frame deterministic subproblems. For each per-frame subproblem, [22] considers the binary offloading decision of a single WD. Likewise, [23] schedules only one user to offload to one of the multiple ESs in each time frame. In both cases, the number of binary offloading variables is very small, and hence the optimal solution can be obtained by brute force search. [24]–[26] consider joint offloading decisions of multiple users. Unlike the binary offloading policy considered in this paper, [24] allows the WDs to process task data in parallel both locally and at the ES, and applies convex optimization to solve the continuous joint offloading and resource allocation problem. In contrast, [25] and [26] adopt binary offloading policy where the number of possible offloading solutions grows exponentially with the user number. To tackle the combinatorial problem, [25] relaxes the binary variables into continuous ones. [26] proposes a two-stage heuristic, which first fixes the resource allocation and then obtains the binary offloading decisions using matching theory. However, these heuristic methods cannot guarantee consistently high solution quality, which may eventually degrade the long-term performance.

In Fig. 2, we illustrate the organization of the rest of the paper. In Section II, we formulate the stable computation offloading problem as a multi-stage stochastic MINLP problem (P1). In Section III, we apply the Lyapunov optimization to decouple (P1) into per-frame deterministic MINLP subproblem (P2). In Section IV, we introduce the LyDROO algorithm to solve (P2) using an actor-critic DRL. The actor module implements a DNN to solve the binary offloading subproblem (P3) and the critic module applies a customized optimization algorithm to solve the continuous resource allocation problem (P4). In Section V, we analyze the performance of the LyDROO algorithm. In Section VI, we evaluate the proposed algorithm via extensive simulations. Finally, we conclude the paper in Section VII.

II. SYSTEM MODEL AND PROBLEM FORMULATION

A. System Model

As shown in Fig. 1, we consider an ES assisting the computation of N WDs in sequential time frames of equal

duration T . Within the t th time frame, we denote A_i^t (in bits) as the raw task data arrival at the data queue of the i th WD. We assume that the arrival A_i^t follows a general i.i.d. distribution with bounded second order moment, i.e., $\mathbb{E}[(A_i^t)^2] = \eta_i < \infty$, for $i = 1, \dots, N$. We assume that the value of η_i is known, e.g., by estimating from past observations. We denote the channel gain between the i th WD and the ES as h_i^t . Under the block fading assumption, h_i^t remains constant within a time frame but varies independently across different frames.

In the t th time frame, suppose that a tagged WD i processes D_i^t bits data and produces a computation output at the end of the time frame. In particular, we assume that the WDs adopt a binary computation offloading rule [2]. That is, within each time frame, the raw data must be processed either locally at the WD or remotely at the ES. For instance, WD 1 and 3 offload their tasks while WD 2 computes locally in Fig. 1. The offloading WDs share a common bandwidth W for transmitting the task data to the ES in a TDMA manner. We use a binary variable x_i^t to denote the offloading decision, where $x_i^t = 1$ and 0 denote that WD i performs computation offloading and local computing, respectively.

When the WD processes the data locally ($x_i^t = 0$), we denote the local CPU frequency as f_i^t , which is upper bounded by f_i^{max} . The raw data (in bits) processed locally and the consumed energy within the time frame are [2]

$$D_{i,L}^t = f_i^t T / \phi, \quad E_{i,L}^t = \kappa (f_i^t)^3 T, \quad \forall x_i^t = 0, \quad (1)$$

respectively. Here, parameter $\phi > 0$ denotes the number of computation cycles needed to process one bit of raw data and $\kappa > 0$ denotes the computing energy efficiency parameter.

Otherwise, when the data is offloaded for edge execution ($x_i^t = 1$), we denote P_i^t as the transmit power constrained by the maximum power $P_i^t \leq P_i^{max}$ and $\tau_i^t T$ as the amount of time allocated to the i th WD for computation offloading. Here, $\tau_i^t \in [0, 1]$ and $\sum_{i=1}^N \tau_i^t \leq 1$. The energy consumed on data offloading is $E_{i,O}^t = P_i^t \tau_i^t T$. Similar to [4] and [8], we neglect the delay on edge computing and result downloading such that the amount of data processed at the edge within the time frame is

$$\begin{aligned} D_{i,O}^t &= \frac{W \tau_i^t T}{v_u} \log_2 \left(1 + \frac{P_i^t h_i^t}{N_0} \right) \\ &= \frac{W \tau_i^t T}{v_u} \log_2 \left(1 + \frac{E_{i,O}^t h_i^t}{\tau_i^t T N_0} \right), \quad \forall x_i^t = 1, \end{aligned} \quad (2)$$

where $v_u \geq 1$ denotes the communication overhead and N_0 denotes the noise power.

Let $D_i^t \triangleq (1 - x_i^t) D_{i,L}^t + x_i^t D_{i,O}^t$ and $E_i^t \triangleq (1 - x_i^t) E_{i,L}^t + x_i^t E_{i,O}^t$ denote the bits computed and energy consumed in time frame t . We define *computation rate* r_i^t and *power consumption* e_i^t in the t th time frame as

$$\begin{aligned} r_i^t &= \frac{D_i^t}{T} = \frac{(1 - x_i^t) f_i^t}{\phi} + x_i^t \frac{W \tau_i^t}{v_u} \log_2 \left(1 + \frac{e_{i,O}^t h_i^t}{\tau_i^t N_0} \right), \\ e_i^t &= \frac{E_i^t}{T} = (1 - x_i^t) \kappa (f_i^t)^3 + x_i^t e_{i,O}^t, \end{aligned} \quad (3)$$

where $e_{i,O}^t \triangleq E_{i,O}^t / T$. For simplicity of exposition, we assume $T = 1$ without loss of generality in the following derivations.

Let $Q_i(t)$ denote the queue length of the i th WD at the beginning of the t th time frame. Then, the queue dynamics can be modeled as

$$Q_i(t+1) = \max \left\{ Q_i(t) - \tilde{D}_i^t + A_i^t, 0 \right\}, \quad i = 1, 2, \dots, \quad (4)$$

where $\tilde{D}_i^t = \min(Q_i(t), D_i^t)$ and $Q_i(1) = 0$. In this paper, we consider infinite queueing capacity for analytical tractability. In the following derivation, we enforce the data causality constraint $D_i^t \leq Q_i(t)$, implying that $Q_i(t) \geq 0$ holds for any t . Thus, the queue dynamics is simplified as

$$Q_i(t+1) = Q_i(t) - D_i^t + A_i^t, \quad i = 1, 2, \dots \quad (5)$$

Definition 1: A discrete time queue $Q_i(t)$ is *strongly stable* if the time average queue length $\lim_{K \rightarrow \infty} \frac{1}{K} \sum_{t=1}^K \mathbb{E}[Q_i(t)] < \infty$, where the expectation is taken with respect to the system random events [21], i.e., channel fading and task data arrivals in this paper.

By the Little's law, the average delay is proportional to the average queue length. Thus, a strongly stable data queue translates to a finite processing delay of each task data bit.

B. Problem Formulation

In this paper, we aim to design an online algorithm to maximize the long-term average weighted sum computation rate of all the WDs under the data queue stability and average power constraints. In particular, we make online decisions in the sense that in each time frame, we optimize the task offloading and the resource allocation decisions for the particular time frame without the assumption of knowing the future realizations of random channel conditions and data arrivals. We denote $\mathbf{x}^t = [x_1^t, \dots, x_N^t]$, $\boldsymbol{\tau}^t = [\tau_1^t, \dots, \tau_N^t]$, $\mathbf{f}^t = [f_1^t, \dots, f_N^t]$ and $\mathbf{e}_O^t = [e_{1,O}^t, \dots, e_{N,O}^t]$, and let $\mathbf{x} = \{\mathbf{x}^t\}_{t=1}^K$, $\boldsymbol{\tau} = \{\boldsymbol{\tau}^t\}_{t=1}^K$, $\mathbf{f} = \{\mathbf{f}^t\}_{t=1}^K$ and $\mathbf{e}_O = \{\mathbf{e}_O^t\}_{t=1}^K$. We formulate the problem as the following multi-stage stochastic MINLP problem (P1):

$$\begin{aligned} & \text{maximize } \lim_{K \rightarrow \infty} \frac{1}{K} \cdot \sum_{t=1}^K \sum_{i=1}^N c_i \tau_i^t \\ & \text{subject to } \sum_{i=1}^N \tau_i^t \leq 1, \quad \forall t, \end{aligned} \quad (6a)$$

$$(1 - x_i^t) f_i^t / \phi + x_i^t \frac{W \tau_i^t}{v_u} \log_2 \left(1 + \frac{e_{i,O}^t h_i^t}{\tau_i^t N_0} \right) \leq Q_i(t), \quad \forall i, t, \quad (6b)$$

$$\begin{aligned} & \lim_{K \rightarrow \infty} \frac{1}{K} \cdot \sum_{t=1}^K \mathbb{E}[Q_i(t)] < \infty, \quad \forall i, \\ & \mathbb{E} \left[(1 - x_i^t) \kappa (f_i^t)^3 + x_i^t e_{i,O}^t \right] \leq \gamma_i, \quad \forall i, \end{aligned} \quad (6c)$$

$$\lim_{K \rightarrow \infty} \frac{1}{K} \cdot \sum_{t=1}^K \mathbb{E}[Q_i(t)] < \infty, \quad \forall i, \quad (6d)$$

$$f_i^t \leq f_i^{\max}, \quad e_{i,O}^t \leq P_i^{\max} \tau_i^t, \quad \forall i, t, \quad (6e)$$

$$x_i^t \in \{0, 1\}, \quad \tau_i^t, f_i^t, e_{i,O}^t \geq 0, \quad \forall i, t. \quad (6f)$$

Here, c_i denotes the fixed weight of the i th WD. (6a) denotes the offloading time constraint. Notice that $\tau_i^t = e_{i,O}^t = 0$ must

hold at the optimum if $x_i^t = 0$. Similarly, $f_i^t = 0$ must hold if $x_i^t = 1$. (6b) corresponds to the data causality constraint. (6c) corresponds to the average power constraint and γ_i is the power threshold. (6d) are the data queue stability constraints. Under the stochastic channels and data arrivals, it is hard to satisfy the long-term constraints when the decisions are made in each time frame without knowing the future realizations of random channel conditions and data arrivals. Besides, the fast-varying channel condition requires real-time decision-making in each short time frame, e.g., within the channel coherence time. In the following, we propose a novel LyDROO framework that solves (P1) with both high robustness and efficiency.

Remark 1: Before leaving this session, we comment on the possible extension of the proposed LyDROO algorithm. (P1) uses a linear utility function $U(r_i^t) = r_i^t$ in the objective. However, we will show later in Section IV that the proposed LyDROO framework is applicable to solve a wide range of problems as long as the resource allocation problem (P4) can be efficiently solved. For instance, we can consider a general non-decreasing concave function $U(r_i^t)$ such that the corresponding (P4) is a convex problem, e.g., α -fairness function $(1 - \alpha)^{-1} (r_i^t)^{1-\alpha}$ with $\alpha \geq 0$ and $\alpha \neq 1$, proportional fairness function $\ln(r_i^t)$, or other suitable QoS (quality of service) utilities (see [27] and the reference therein). For analytical clarity, we consider in this paper a specific linear utility function to highlight the features of the LyDROO framework.

III. LYAPUNOV-BASED DECOUPLING OF THE MULTI-STAGE MINLP

In this section, we apply the Lyapunov optimization to decouple (P1) into per-frame deterministic problems. To cope with the average power constraints (6c), we introduce N virtual energy queues $\{Y_i(t)\}_{i=1}^N$, one for each WD. Specifically, we set $Y_i(1) = 0$ and update the queue as

$$Y_i(t+1) = \max(Y_i(t) + \nu e_i^t - \nu \gamma_i, 0), \quad (7)$$

for $i = 1, \dots, N$ and $t = 1, \dots, K$, where e_i^t in (3) is the energy consumption at the t th time frame and ν is a positive scaling factor. $Y_i(t)$ can be viewed as a queue with random "energy arrivals" νe_i^t and fixed "service rate" $\nu \gamma_i$. Intuitively, when the virtual energy queues are stable, the average power consumption e_i^t (i.e., the virtual queue arrival rate) does not exceed γ_i , and thus the constraints in (6c) are satisfied.

To jointly control the data and energy queues, we define $\mathbf{Z}(t) = \{\mathbf{Q}(t), \mathbf{Y}(t)\}$ as the total queue backlog, where $\mathbf{Q}(t) = \{Q_i(t)\}_{i=1}^N$ and $\mathbf{Y}(t) = \{Y_i(t)\}_{i=1}^N$. Then, we introduce the Lyapunov function $L(\mathbf{Z}(t))$ and Lyapunov drift $\Delta L(\mathbf{Z}(t))$ as [21]

$$\begin{aligned} L(\mathbf{Z}(t)) &= 0.5 \left(\sum_{i=1}^N Q_i(t)^2 + \sum_{i=1}^N Y_i(t)^2 \right), \\ \Delta L(\mathbf{Z}(t)) &= \mathbb{E} \{ L(\mathbf{Z}(t+1)) - L(\mathbf{Z}(t)) | \mathbf{Z}(t) \}. \end{aligned} \quad (8)$$

To maximize the time average computation rate while stabilizing the queue $\mathbf{Z}(t)$, we use the drift-plus-penalty minimization approach [28]. Specifically, we seek to minimize an upper

bound on the following drift-plus-penalty expression at every time frame t :

$$\Lambda(\mathbf{Z}(t)) \triangleq \Delta L(\mathbf{Z}(t)) - V \cdot \sum_{i=1}^N \mathbb{E} \{c_i r_i^t | \mathbf{Z}(t)\}, \quad (9)$$

where $V > 0$ is an “importance” weight to scale the penalty.

In the following, we derive an upper bound of $\Lambda(\mathbf{Z}(t))$. To begin with, we have

$$\begin{aligned} Q_i(t+1)^2 &= Q_i(t)^2 + 2Q_i(t)(A_i^t - D_i^t) + (A_i^t - D_i^t)^2, \\ Y_i(t+1)^2 &= Y_i(t)^2 + 2Y_i(t)(e_i^t - \gamma_i) + (e_i^t - \gamma_i)^2. \end{aligned}$$

By taking the sum over the N queues on both sides, we have

$$\begin{aligned} 0.5 \sum_{i=1}^N Q_i(t+1)^2 - 0.5 \sum_{i=1}^N Q_i(t)^2 \\ = 0.5 \sum_{i=1}^N (A_i^t - D_i^t)^2 + \sum_{i=1}^N Q_i(t)(A_i^t - D_i^t) \end{aligned} \quad (10)$$

and

$$\begin{aligned} 0.5 \sum_{i=1}^N Y_i(t+1)^2 - 0.5 \sum_{i=1}^N Y_i(t)^2 \\ = 0.5 \sum_{i=1}^N (e_i^t - \gamma_i)^2 + \sum_{i=1}^N Y_i(t)(e_i^t - \gamma_i). \end{aligned} \quad (11)$$

We define

$$L(\mathbf{Q}(t)) \triangleq 0.5 \sum_{i=1}^N Q_i(t)^2 \quad (12)$$

and

$$\Delta L(\mathbf{Q}(t)) \triangleq \mathbb{E} \{L(\mathbf{Q}(t+1)) - L(\mathbf{Q}(t)) | \mathbf{Z}(t)\}. \quad (13)$$

By taking the conditional expectation on both sides of (10), we have

$$\Delta L(\mathbf{Q}(t)) \leq B_1 + \sum_{i=1}^N Q_i(t) \mathbb{E} [(A_i^t - D_i^t) | \mathbf{Z}(t)]. \quad (14)$$

Here, B_1 is a constant obtained as

$$\begin{aligned} 0.5 \sum_{i=1}^N \mathbb{E} [(A_i^t - D_i^t)^2] &\leq 0.5 \sum_{i=1}^N \mathbb{E} [(A_i^t)^2 + (D_i^t)^2] \\ &\leq 0.5 \sum_{i=1}^N (\eta_i + [T \max \{f_i^{max}/\phi, r_i^{max}\}]^2) \triangleq B_1, \end{aligned}$$

where the second inequality holds because $r_i^{max} \triangleq \mathbb{E} \left[\frac{W}{v_u} \log_2 \left(1 + \frac{P_i^{max} h_i^t}{N_0} \right) \right]$ corresponds to the maximum average transmission rate of the i th WD.

Similarly, we define

$$L(\mathbf{Y}(t)) = 0.5 \sum_{i=1}^N Y_i(t)^2 \quad (15)$$

and

$$\Delta L(\mathbf{Y}(t)) \triangleq \mathbb{E} \{L(\mathbf{Y}(t+1)) - L(\mathbf{Y}(t)) | \mathbf{Z}(t)\}. \quad (16)$$

We obtain the following by taking the expectation on both sides of (11)

$$\Delta L(\mathbf{Y}(t)) \leq B_2 + \sum_{i=1}^N Y_i(t) \mathbb{E} [e_i^t - \gamma_i | \mathbf{Z}(t)], \quad (17)$$

where the constant B_2 is obtained from

$$\begin{aligned} 0.5 \sum_{i=1}^N \mathbb{E} [(e_i^t - \gamma_i)^2] \\ \leq 0.5 \sum_{i=1}^N \left[\left(\max \left\{ \kappa (f_i^{max})^3, P_i^{max} \right\} \right)^2 + \gamma_i^2 \right] \triangleq B_2. \end{aligned}$$

Summing over the two inequalities in (14) and (17), we have

$$\begin{aligned} \Delta L(\mathbf{Z}(t)) &\leq \hat{B} + \sum_{i=1}^N Q_i(t) \mathbb{E} [(A_i^t - D_i^t) | \mathbf{Z}(t)] \\ &\quad + \sum_{i=1}^N Y_i(t) \mathbb{E} [e_i^t - \gamma_i | \mathbf{Z}(t)] \end{aligned} \quad (18)$$

where $\hat{B} = B_1 + B_2$. Therefore, the upper bound of the drift-plus-penalty expression in (9) is

$$\begin{aligned} \hat{B} + \sum_{i=1}^N \{Q_i(t) \mathbb{E} [(A_i^t - D_i^t) | \mathbf{Z}(t)] \\ + Y_i(t) \mathbb{E} [e_i^t - \gamma_i | \mathbf{Z}(t)] - V \mathbb{E} [c_i r_i^t | \mathbf{Z}(t)]\}. \end{aligned} \quad (19)$$

In the t th time frame, we apply the technique of *opportunistic expectation minimization* [21]. That is, we observe the queue backlogs $\mathbf{Z}(t)$ and decide the joint offloading and resource allocation control action accordingly to minimize the upper bound in (19). Notice that only the second term is related to the control action in the t th time frame. By removing the constant terms from the observation at the beginning of the t th time frame, the algorithm decides the actions by maximizing the following:

$$\sum_{i=1}^N (Q_i(t) + V c_i) r_i^t - \sum_{i=1}^N Y_i(t) e_i^t, \quad (20)$$

where r_i^t and e_i^t are in (3). Intuitively, it tends to increase the computation rates of WDs that have a long data queue backlog or a large weight, while penalizing those that have exceeded the average power threshold. We introduce an auxiliary variable $r_{i,O}^t$ for each WD i and denote $\mathbf{r}_O^t = \{r_{i,O}^t\}_{i=1}^N$. Taking into account the per-frame constraints, we solve the following deterministic per-frame subproblem (P2) in the t th time frame

$$\begin{aligned} &\text{maximize}_{\mathbf{x}^t, \mathbf{r}^t, \mathbf{f}^t, \mathbf{e}_O^t, \mathbf{r}_O^t} \sum_{i=1}^N (Q_i(t) + V c_i) r_i^t - \sum_{i=1}^N Y_i(t) e_i^t \\ &\text{subject to} \sum_{i=1}^N \tau_i^t \leq 1, \end{aligned} \quad (21a)$$

$$f_i^t / \phi \leq Q_i(t), \quad r_{i,O}^t \leq Q_i(t), \quad \forall i, \quad (21b)$$

$$r_{i,O}^t \leq \frac{W \tau_i^t}{v_u} \log_2 \left(1 + \frac{e_{i,O}^t h_i^t}{\tau_i^t N_0} \right), \quad \forall i, \quad (21c)$$

$$f_i^t \leq f_i^{max}, \quad e_{i,O}^t \leq P_i^{max} \tau_i^t, \quad \forall i, \quad (21d)$$

$$x_i^t \in \{0, 1\}, \quad \tau_i^t, f_i^t, e_{i,O}^t \geq 0, \quad \forall i. \quad (21e)$$

Notice that the above constraints (21b) and (21c) are equivalent to (6b) in (P1), because there is exactly one non-zero term in the left-hand side of (6b) at the optimum. In Section V, we will show that we can satisfy all long-term constraints in (P1) by solving the per-frame subproblems in an online fashion. Then, the remaining difficulty lies in solving the MINLP (P2) in each time frame. In the following section, we propose a DRL-based algorithm to solve (P2) efficiently.

IV. LYAPUNOV-GUIDED DRL FOR ONLINE COMPUTATION OFFLOADING

Recall that to solve (P2) in the t th time frame, we observe $\xi^t \triangleq \{h_i^t, Q_i(t), Y_i(t)\}_{i=1}^N$, consisting of the channel gains $\{h_i^t\}_{i=1}^N$ and the system queue states $\{Q_i(t), Y_i(t)\}_{i=1}^N$, and accordingly decide the control action $\{\mathbf{x}^t, \mathbf{y}^t\}$, including the binary offloading decision $\mathbf{x}^t$ and the continuous resource allocation $\mathbf{y}^t \triangleq \{\tau_i^t, f_i^t, e_{i,O}^t, r_{i,O}^t\}_{i=1}^N$. A close observation shows that although (P2) is a non-convex optimization problem, the resource allocation problem to optimize $\mathbf{y}^t$ is in fact an “easy” convex problem if $\mathbf{x}^t$ is fixed. In Section IV.B, we will propose a customized algorithm to efficiently obtain the optimal $\mathbf{y}^t$ given $\mathbf{x}^t$ in (P2). Here, we denote $G(\mathbf{x}^t, \xi^t)$

as the optimal value of (P2) by optimizing $\mathbf{y}^t$ given the offloading decision $\mathbf{x}^t$ and parameter ξ^t . Therefore, solving (P2) is equivalent to finding the optimal offloading decision $(\mathbf{x}^t)^*$, where

$$(P3) : (\mathbf{x}^t)^* = \arg \max_{\mathbf{x}^t \in \{0,1\}^N} G(\mathbf{x}^t, \xi^t). \quad (22)$$

In general, obtaining $(\mathbf{x}^t)^*$ requires enumerating 2^N offloading decisions, which leads to significantly high computational complexity even when N is moderate (e.g., $N = 10$). Other search based methods, such as branch-and-bound and block coordinate descent [29], are also time-consuming when N is large. In practice, neither method is applicable to online decision-making under fast-varying channel condition. Leveraging the DRL technique, we propose a LyDROO algorithm to construct a policy π that maps from the input ξ^t to the optimal action $(\mathbf{x}^t)^*$, i.e., $\pi : \xi^t \mapsto (\mathbf{x}^t)^*$, with very low complexity, e.g., tens of milliseconds computation time (i.e., the time duration from observing ξ^t to producing a control action $\{\mathbf{x}^t, \mathbf{y}^t\}$) when $N = 10$.

A. Algorithm Description

As illustrated in Fig. 3, LyDROO consists of four main modules: an actor module that accepts the input ξ^t and outputs a set of candidate offloading actions $\{\mathbf{x}_i^t\}$, a critic module evaluates $\{\mathbf{x}_i^t\}$ and selects the best offloading action $\mathbf{x}^t$, a policy update module improves the policy of the actor module over time, and a queueing module updates the system queue states $\{Q_i(t), Y_i(t)\}_{i=1}^N$ after executing the offloading actions. Through repeated interactions with the random environment $\{h_i^t, A_i^t\}_{i=1}^N$, the four modules operate in a sequential and iterative manner as detailed below.

1) *Actor Module*: The actor module consists of a DNN and an action quantizer. At the beginning of the t th time frame, we denote the parameter of the DNN as θ^t , which is randomly initialized following the standard normal distribution when $t = 1$. Taking the observation ξ^t as the input, the DNN outputs a relaxed offloading decision $\hat{\mathbf{x}}^t \in [0, 1]^N$ that is later to be quantized into feasible binary actions. The input-output relation is expressed as

$$\Pi_{\theta^t} : \xi^t \mapsto \hat{\mathbf{x}}^t = \{\hat{x}_i^t \in [0, 1], i = 1, \dots, N\}. \quad (23)$$

The well-known universal approximation theorem claims that a multi-layer perceptron with a sufficient number of neurons can accurately approximate any continuous mappings if proper activation functions are applied at the neurons, e.g., sigmoid, ReLu, and tanh functions [30]. Here, we use a sigmoid activation function at the output layer.

We then quantize the continuous $\hat{\mathbf{x}}^t$ into M_t feasible candidate binary offloading actions, where M_t is a time-dependent design parameter. The quantization function is expressed as:

$$\Upsilon_{M_t} : \hat{\mathbf{x}}^t \mapsto \Omega^t = \{\mathbf{x}_j^t | \mathbf{x}_j^t \in \{0, 1\}^N, j = 1, \dots, M_t\}, \quad (24)$$

where Ω_t denotes the set of candidate offloading actions in the t th time frames. Υ_{M_t} represents a quantization function that generates $M_t = |\Omega_t|$ binary actions. A good quantization function should balance the *exploration-exploitation tradeoff*

in generating the offloading action to ensure good training convergence. Intuitively, $\{\mathbf{x}_j^t\}$'s should be close to $\hat{\mathbf{x}}^t$ (measured by Euclidean distance) to make effective use of the DNN's output and meanwhile sufficiently separate to avoid premature convergence to sub-optimal solution in the training process.

Here, we apply the noisy order-preserving (NOP) quantization method [31], which can generate any $M_t \leq 2N$ candidate actions. The NOP method generates the first $M_t/2$ actions (M_t is assumed an even number) by applying the order-preserving quantizer (OPQ) in [18] to $\hat{\mathbf{x}}^t$. Specifically, the 1st action $\mathbf{x}_1^t = [x_{1,1}^t, \dots, x_{1,N}^t]$ is calculated as

$$x_{1,i}^t = \begin{cases} 1 & \hat{x}_i^t > 0.5, \\ 0 & \hat{x}_i^t \leq 0.5, \end{cases} \quad (25)$$

for $i = 1, \dots, N$. To generate the next $M_t/2 - 1$ actions, we order the entries of $\hat{\mathbf{x}}^t$ based on the distance to 0.5, such that $|\hat{x}_{(1)}^t - 0.5| \leq |\hat{x}_{(2)}^t - 0.5| \leq \dots \leq |\hat{x}_{(i)}^t - 0.5| \leq \dots \leq |\hat{x}_{(N)}^t - 0.5|$, where $\hat{x}_{(i)}^t$ denotes the i th ordered entry of $\hat{\mathbf{x}}^t$. Then, $\hat{x}_{(i)}^t$'s are used as the decision thresholds to quantize $\hat{\mathbf{x}}^t$, where the m th action $\mathbf{x}_m^t$, for $m = 2, \dots, M_t/2$, is obtained from entry-wise comparisons of $\hat{\mathbf{x}}^t$ and $\hat{x}_{(m-1)}^t$. That is,

$$x_{m,i}^t = \begin{cases} 1, & \hat{x}_i^t > \hat{x}_{(m-1)}^t \\ \text{or } \left\{ \hat{x}_i^t = \hat{x}_{(m-1)}^t \text{ and } \hat{x}_{(m-1)}^t \leq 0.5 \right\}, & \\ 0, & \hat{x}_i^t < \hat{x}_{(m-1)}^t \\ \text{or } \left\{ \hat{x}_i^t = \hat{x}_{(m-1)}^t \text{ and } \hat{x}_{(m-1)}^t > 0.5 \right\}, & \end{cases} \quad (26)$$

for $i = 1, \dots, N$. To obtain the remaining $M_t/2$ actions, we first generate a noisy version of $\hat{\mathbf{x}}^t$ denoted as $\tilde{\mathbf{x}}^t = \text{Sigmoid}(\hat{\mathbf{x}}^t + \mathbf{n})$, where the random Gaussian noise $\mathbf{n} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_N)$ with $\mathbf{I}_N$ being an identity matrix, and $\text{Sigmoid}(\cdot)$ is the element-wise Sigmoid function that bounds each entry of $\tilde{\mathbf{x}}^t$ within $(0, 1)$. Then, we produce the remaining $M_t/2$ actions $\mathbf{x}_m^t$, for $m = M_t/2 + 1, \dots, M_t$, by applying the OPQ to $\tilde{\mathbf{x}}^t$, i.e., replacing $\hat{\mathbf{x}}^t$ with $\tilde{\mathbf{x}}^t$ in (25) and (26).

2) *Critic Module*: Followed by the actor module, the critic module evaluates $\{\mathbf{x}_i^t\}$ and selects the best offloading action $\mathbf{x}^t$. Unlike the conventional actor-critic structure that uses a model-free DNN as the critic network to evaluate the action, LyDROO leverages the model information to evaluate the binary offloading action by analytically solving the optimal resource allocation problem. This enables the critic module to have accurate evaluation of the offloading actions, and thus achieving more robust and faster convergence of the DRL training process.

Specifically, LyDROO selects the best action $\mathbf{x}^t$ as

$$\mathbf{x}^t = \arg \max_{\mathbf{x}_j^t \in \Omega_t} G(\mathbf{x}_j^t, \xi^t), \quad (27)$$

where $G(\mathbf{x}_j^t, \xi^t)$ is obtained by optimizing the resource allocation given $\mathbf{x}_j^t$ in (P2). We will introduce the detailed algorithm to obtain $G(\mathbf{x}_j^t, \xi^t)$ in Section IV.B. Notice that the calculation of $G(\mathbf{x}_j^t, \xi^t)$ is performed by M_t times to obtain the best action $\mathbf{x}^t$. Intuitively, a larger M_t results in better solution performance, but a larger computation time. To balance the performance-complexity tradeoff, we propose here an adaptive procedure to set a time-varying M_t .

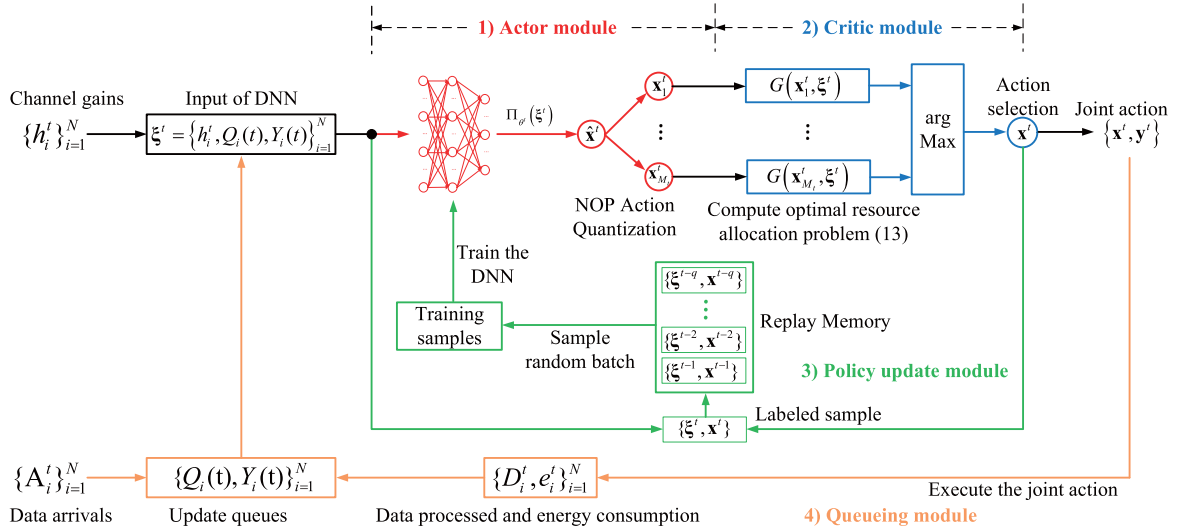


Fig. 3. The schematics of the proposed LyDROO algorithm.

The key idea is that when the actor DNN gradually approaches the optimal policy over time, a small M_t suffices to find the optimal action within a small distance to $\hat{\mathbf{x}}^t$. Denote $m_t \in [0, M_t - 1]$ as the index of the best action $\mathbf{x}^t \in \Omega_t$. We define $m_t^* = \text{mod}(m_t, M_t/2)$, which represents the order of $\mathbf{x}^t$ among either the $M_t/2$ noise-free or the noise-added candidate actions. In practice, we set a maximum $M_1 = 2N$ initially and update M_t every $\delta_M \geq 1$ time frames. If $\text{mod}(t, \delta_M) = 0$ in time frame t , i.e., t can be divided by δ_M , we set

$$M_t = 2 \cdot \min(\max(m_{t-1}^*, \dots, m_{t-\delta_M}^*) + 1, N). \quad (28)$$

The additional 1 in the first term within the min operator allows M_t to increase over time. Otherwise, $M_t = M_{t-1}$ if $\text{mod}(t, \delta_M) \neq 0$. Notice that too frequent update (small δ_M) may degrade the training convergence while a too large δ_M cause unnecessary computational complexity.

3) *Policy Update Module*: LyDROO uses $(\xi^t, \mathbf{x}^t)$ as a labeled input-output sample for updating the policy of the DNN. In particular, we maintain a replay memory that only stores the most recent q data samples. In practice, with an initially empty memory, we start training the DNN after collecting more than $q/2$ data samples. Then, the DNN is trained periodically once every δ_T time slots to avoid model over-fitting. When $\text{mod}(t, \delta_T) = 0$, we randomly select a batch of data samples $\{(\xi^\tau, \mathbf{x}^\tau) | \tau \in \mathcal{S}^t\}$, where $\mathcal{S}^t$ denotes the set of time indices of the selected samples. We then update the parameter of the DNN by minimizing its average cross-entropy loss function $LS(\theta^t)$ over the data samples using the Adam algorithm [30]

$$LS(\theta^t) = -1/|\mathcal{S}^t| \cdot \sum_{\tau \in \mathcal{S}^t} [(\mathbf{x}^\tau)^\top \log \Pi_{\theta^t}(\xi^\tau) + (1 - \mathbf{x}^\tau)^\top \log(1 - \Pi_{\theta^t}(\xi^\tau))], \quad (29)$$

where $|\mathcal{S}^t|$ denotes the size of the sample batch, $(\cdot)^\top$ denotes the transpose operator, and the log function denotes the element-wise logarithm operation of a vector. When the training completes, we update the parameter of the actor module in the next time frame to θ^{t+1} .

Algorithm 1 The Online LyDROO Algorithm for Solving (P1)

input : Parameters $V, \{\gamma_i, c_i\}_{i=1}^N, K$, training interval δ_T, M_t update interval δ_M ;
output: Control actions $\{\mathbf{x}^t, \mathbf{y}^t\}_{t=1}^K$;
1 Initialize the DNN with random parameters θ^1 and empty replay memory, $M_1 \leftarrow 2N$;
2 Empty initial data queue $Q_i(1) = 0$ and energy queue $Y_i(1) = 0$, for $i = 1, \dots, N$;
3 **for** $t = 1, 2, \dots, K$ **do**
4 Observe the input $\xi^t = \{h^t, Q_i(t), Y_i(t)\}_{i=1}^N$ and update M_t using (8) if $\text{mod}(t, \delta_M) = 0$;
5 Generate a relaxed offloading action $\hat{\mathbf{x}}^t = \Pi_{\theta^t}(\xi^t)$ with the DNN;
6 Quantize $\hat{\mathbf{x}}^t$ into M_t binary actions $\{\mathbf{x}_i^t | i = 1, \dots, M_t\}$ using the NOP method;
7 Compute $G(\mathbf{x}_i^t, \xi^t)$ by optimizing resource allocation $\mathbf{y}_i^t$ in (P2) for each $\mathbf{x}_i^t$;
8 Select the best solution $\mathbf{x}^t = \arg \max_{\{\mathbf{x}_i^t\}} G(\mathbf{x}_i^t, \xi^t)$ and execute the joint action $(\mathbf{x}^t, \mathbf{y}^t)$;
9 Update the replay memory by adding $(\xi^t, \mathbf{x}^t)$;
10 **if** $\text{mod}(t, \delta_T) = 0$ **then**
11 Uniformly sample a batch of data set $\{(\xi^\tau, \mathbf{x}^\tau) | \tau \in \mathcal{S}^t\}$ from the memory;
12 Train the DNN with $\{(\xi^\tau, \mathbf{x}^\tau) | \tau \in \mathcal{S}^t\}$ and update θ^t using the Adam algorithm;
13 **end**
14 $t \leftarrow t + 1$;
15 Update $\{Q_i(t), Y_i(t)\}_{i=1}^N$ based on $(\mathbf{x}^{t-1}, \mathbf{y}^{t-1})$ and data arrival observation $\{A_i^{t-1}\}_{i=1}^N$ using (5) and (7).
16 **end**

4) *Queueing Module*: As a by-product of the critic module, we obtain the optimal resource allocation $\mathbf{y}^t$ associated with $\mathbf{x}^t$. Accordingly, the system executes the joint computation offloading and resource allocation action $\{\mathbf{x}^t, \mathbf{y}^t\}$, which

processes data $\{D_i^t\}_{i=1}^N$ and consumes energy $\{e_i^t\}_{i=1}^N$ as given in (3). Based on $\{D_i^t, e_i^t\}_{i=1}^N$ and the data arrivals $\{A_i^t\}_{i=1}^N$ observed in the t th time frame, the queueing module then updates the data and energy queues $\{Q_i(t+1), Y_i(t+1)\}_{i=1}^N$ using (5) and (7) at the beginning of the $(t+1)$ th time frame. With the wireless channel gains observation $\{h_i^{t+1}\}_{i=1}^N$, the system feeds the input parameter $\xi^{t+1} = \{h_i^{t+1}, Q_i(t+1), Y_i(t+1)\}_{i=1}^N$ to the DNN and starts a new iteration from the actor module in Step 1).

With the above actor-critic-update loop, the DNN consistently learns from the best and most recent state-action pairs, leading to a better policy π_{θ^t} that gradually approximates the optimal mapping to solve (P3). We summarize the pseudo-code of LyDROO in Algorithm 1, where the major computational complexity is in line 7 that computes $G(\mathbf{x}_i^t, \xi^t)$ by solving the optimal resource allocation problems. This in fact indicates that the proposed LyDROO algorithm can be extended to solve (P1) when considering a general non-decreasing concave utility $U(r_i^t)$ in the objective, because the per-frame resource allocation problem to compute $G(\mathbf{x}_i^t, \xi^t)$ is a convex problem that can be efficiently solved, where the detailed analysis is omitted. In the next subsection, we propose a low-complexity algorithm to obtain $G(\mathbf{x}_i^t, \xi^t)$.

B. Low-Complexity Optimal Resource Allocation Algorithm

Given the value of $\mathbf{x}^t$ in (P2), we denote the index set of users with $x_i^t = 1$ as $\mathcal{M}_1^t$, and the complementary user set as $\mathcal{M}_0^t$. For simplicity of exposition, we drop the superscript t and express the optimal resource allocation problem that computes $G(\mathbf{x}^t, \xi^t)$ as following

$$\begin{aligned} \text{(P4): } & \underset{\tau, f, e_{i,O}, r_{i,O}}{\text{maximize}} \sum_{j \in \mathcal{M}_0} \{a_j f_j / \phi - Y_j(t) \kappa f_j^3\} \\ & + \sum_{i \in \mathcal{M}_1} \{a_i r_{i,O} - Y_i(t) e_{i,O}\} \\ \text{subject to } & f_j / \phi \leq Q_j(t), 0 \leq f_j \leq f_j^{\max}, \quad \forall j \in \mathcal{M}_0, \\ & \sum_{i \in \mathcal{M}_1} \tau_i \leq 1, \\ & e_{i,O} \leq P_i^{\max} \tau_i, r_{i,O} \leq Q_i(t), \quad \forall i \in \mathcal{M}_1, \\ & r_{i,O} \leq \frac{W \tau_i}{v_u} \log_2 \left(1 + \frac{e_{i,O} h_i}{\tau_i N_0} \right), \quad \forall i \in \mathcal{M}_1, \\ & \tau_i, r_{i,O}, e_{i,O} \geq 0, \quad \forall i \in \mathcal{M}_1, \end{aligned}$$

where $a_i \triangleq Q_i(t) + V c_i$ is a parameter. Notice that (P4) can be separately optimized for WDs in $\mathcal{M}_1$ and $\mathcal{M}_0$. In particular, each $j \in \mathcal{M}_0$ solves an independent problem

$$\begin{aligned} & \underset{f_j}{\text{maximize}} \quad a_j f_j / \phi - Y_j(t) \kappa f_j^3 \\ & \text{subject to } 0 \leq f_j \leq \min \{ \phi Q_j(t), f_j^{\max} \}, \end{aligned} \quad (30)$$

where the closed-form optimal solution is

$$f_j^* = \min \left\{ \sqrt{\frac{a_j}{3 \phi \kappa Y_j(t)}}, \min \{ \phi Q_j(t), f_j^{\max} \} \right\}, \quad \forall j \in \mathcal{M}_0. \quad (31)$$

Intuitively, the j th WD computes faster when $Q_j(t)$ is large or $Y_j(t)$ is small, and vice versa.

On the other hand, denote $\hat{\tau} = \{\tau_i, \forall i \in \mathcal{M}_1\}$, $\hat{e}_O = \{e_{i,O}, \forall i \in \mathcal{M}_1\}$ and $\hat{r}_O = \{r_{i,O}, \forall i \in \mathcal{M}_1\}$, we need to

solve the following problem for the WDs in $\mathcal{M}_1$,

$$\begin{aligned} & \underset{\hat{\tau}, \hat{e}_O, \hat{r}_O}{\text{maximize}} \sum_{i \in \mathcal{M}_1} \{a_i r_{i,O} - Y_i(t) e_{i,O}\} \\ & \text{subject to } \sum_{i \in \mathcal{M}_1} \tau_i \leq 1, \\ & e_{i,O} \leq P_i^{\max} \tau_i, r_{i,O} \leq Q_i(t), \quad \forall i \in \mathcal{M}_1, \\ & r_{i,O} \leq \frac{W \tau_i}{v_u} \log_2 \left(1 + \frac{e_{i,O} h_i}{\tau_i N_0} \right), \quad \forall i \in \mathcal{M}_1, \\ & \tau_i, r_{i,O}, e_{i,O} \geq 0, \quad \forall i \in \mathcal{M}_1. \end{aligned}$$

We express a partial Lagrangian of the problem as

$$L(\{\hat{\tau}, \hat{e}_O, \hat{r}_O\}, \mu) = \sum_{i \in \mathcal{M}_1} \{a_i r_{i,O} - Y_i(t) e_{i,O}\} + \mu (1 - \sum_{i \in \mathcal{M}_1} \tau_i), \quad (33)$$

where μ denotes the dual variable. Furthermore, the dual function is

$$\begin{aligned} d(\mu) &= \underset{\hat{\tau}, \hat{e}_O, \hat{r}_O}{\text{maximize}} \quad L(\{\hat{\tau}, \hat{e}_O, \hat{r}_O\}, \mu) \\ & \text{subject to } e_{i,O} \leq P_i^{\max} \tau_i, r_{i,O} \leq Q_i(t), \quad \forall i \in \mathcal{M}_1, \\ & r_{i,O} \leq \frac{W \tau_i}{v_u} \log_2 \left(1 + \frac{e_{i,O} h_i}{\tau_i N_0} \right), \quad \forall i \in \mathcal{M}_1, \\ & \tau_i, r_{i,O}, e_{i,O} \geq 0, \quad \forall i \in \mathcal{M}_1 \end{aligned}$$

and the dual problem is minimize $d(\mu)$. Notice that the dual function can be decomposed into parallel sub-problems. For a WD $i \in \mathcal{M}_1$, it solves

$$\underset{\tau_i, e_{i,O}, r_{i,O}}{\text{maximize}} \{a_i r_{i,O} - Y_i(t) e_{i,O}\} - \mu \tau_i \quad (34a)$$

$$\text{subject to } \tau_i \geq 0, 0 \leq e_{i,O} \leq P_i^{\max} \tau_i, \quad (34b)$$

$$0 \leq r_{i,O} \leq Q_i(t), \quad (34c)$$

$$r_{i,O} \leq \frac{W \tau_i}{v_u} \log_2 \left(1 + \frac{e_{i,O} h_i}{\tau_i N_0} \right). \quad (34d)$$

In the following, we propose a simple algorithm that solves (34) efficiently.

Notice that equality (34d) holds at the optimum because otherwise we can reduce the value of $e_{i,O}$ at the optimum to achieve a higher objective. By setting $r_{i,O} = \frac{W \tau_i}{v_u} \log_2 \left(1 + \frac{e_{i,O} h_i}{\tau_i N_0} \right)$ in (34d), we can equivalently write the constraint $0 \leq e_{i,O} \leq P_i^{\max} \tau_i$ in (34b) as

$$0 \leq \frac{r_{i,O}}{\tau_i} \leq \frac{W}{v_u} \log_2 \left(1 + \frac{P_i^{\max} h_i}{N_0} \right) \triangleq R_i^{\max}, \quad (35)$$

where $R_i^{\max}$ denotes the maximum transmission rate of the i th WD. From (3), we express $e_{i,O}$ as a function of $r_{i,O}$ and τ_i as

$$e_{i,O} = \frac{N_0 \tau_i}{h_i} \left(2^{\frac{r_{i,O} v_u}{W \tau_i}} - 1 \right) \triangleq \frac{\tau_i}{h_i} g \left(\frac{r_{i,O}}{\tau_i} \right), \quad (36)$$

where $g(x) \triangleq N_0 \left(2^{\frac{x v_u}{W}} - 1 \right)$ is a convex function. By plugging (35) and (36) into (34), we can equivalently transform (34) as the following problem

$$\begin{aligned} & \underset{r_{i,O}, \tau_i}{\text{maximize}} \quad a_i r_{i,O} - \mu \tau_i - Y_i(t) \frac{\tau_i}{h_i} g \left(\frac{r_{i,O}}{\tau_i} \right) \\ & \text{subject to } \tau_i \geq r_{i,O} / R_i^{\max}, 0 \leq r_{i,O} \leq Q_i(t). \end{aligned} \quad (37)$$

Notice that (37), and thus (34), is equivalent to the following problem

$$\underset{r_{i,O}}{\text{maximize}} \{V_i(r_{i,O}) | 0 \leq r_{i,O} \leq Q_i(t)\}, \quad (38)$$

where

$$\begin{aligned} V_i(r_{i,O}) \triangleq & \underset{\tau_i}{\text{maximize}} a_i r_{i,O} - \mu \tau_i - Y_i(t) \frac{\tau_i}{h_i} g\left(\frac{r_{i,O}}{\tau_i}\right) \\ & \text{subject to } \tau_i \geq r_{i,O} / R_i^{\max}. \end{aligned} \quad (39)$$

(39) is a convex problem, where we derive the optimal solution in the following Proposition 1.

Proposition 1: The optimal solution of (39) is

$$\tau_i^* = \begin{cases} \frac{r_{i,O}}{R_i^{\max}}, & \text{if } h_i \leq \psi_i(\mu), \\ \frac{\ln 2v_u \cdot r_{i,O}}{W \cdot \left[\mathcal{W}\left(e^{-1} \left[\frac{\mu h_i}{Y_i(t) N_0} - 1 \right] + 1 \right) \right] + 1}, & \text{otherwise,} \end{cases} \quad (40)$$

where $\psi_i(\mu) \triangleq \frac{N_0}{P_i^{\max}} \left(\frac{A_i}{-\mathcal{W}(-A_i \exp(-A_i))} - 1 \right)$ and $A_i \triangleq 1 + \frac{\mu}{Y_i(t) P_i^{\max}}$ are fixed parameters given μ . $\mathcal{W}(x)$ denotes the Lambert-W function, which is the inverse function of $J(z) = z \exp(z) = x$, i.e., $z = \mathcal{W}(x)$.

Proof: Please see the detailed proof in the Appendix A. ■

Remark 2: A close observation of (40) shows that we can compactly express the optimal solution τ_i^* of problem (39) as $r_{i,O} = l_i(\mu) \tau_i^*$, where $l_i(\mu)$ is a fixed parameter given μ , representing the optimal communication data rate of the i th WD. In other words, the optimal transmission time τ_i^* of (39) increases linearly with $r_{i,O}$ under a fixed transmission rate $l_i(\mu)$. In the following, we show how to obtain the dual optimal solution μ^* and retrieve the primal optimal solutions to (32) accordingly.

From Proposition 1, we plug $r_{i,O} = l_i(\mu) \tau_i$ into (39) and rewrite problem (38) as

$$\begin{aligned} & \underset{r_{i,O}}{\text{maximize}} \left\{ a_i - \frac{\mu}{l_i(\mu)} - Y_i(t) \frac{g[l_i(\mu)]}{l_i(\mu) h_i} \right\} r_{i,O} \\ & \text{subject to } 0 \leq r_{i,O} \leq Q_i(t), \end{aligned} \quad (41)$$

where the optimal solution is

$$r_{i,O}^* = \begin{cases} 0, & \text{if } a_i - \frac{\mu}{l_i(\mu)} - Y_i(t) \frac{g[l_i(\mu)]}{l_i(\mu) h_i} < 0, \\ Q_i(t), & \text{otherwise.} \end{cases} \quad (42)$$

Accordingly, we have $\tau_i^* = r_{i,O}^* / l_i(\mu)$. After obtaining τ_i^* , $\forall i \in \mathcal{M}_1$, we calculate the subgradient of μ in (33) as $1 - \sum_{i \in \mathcal{M}_1} \tau_i^*$. Then, we obtain the optimal dual variable μ^* through the ellipsoid method (bi-section search in this case) over the range $[0, \Delta]$, where Δ is a sufficiently large value, until a prescribed precision requirement is met.

Given the optimal μ^* , we denote the optimal ratio obtained from (40) as $l_i(\mu^*) \triangleq r_{i,O}^* / \tau_i^*$, $\forall i \in \mathcal{M}_1$. Notice that the optimal solution $\{\tau_i^*, r_{i,O}^*, \forall i \in \mathcal{M}_1\}$ of the dual problem may not be primal feasible. Therefore, to find a primal optimal solution to (32), we substitute $\tau_i = r_{i,O} / l_i(\mu^*)$ into (32) and simplify the problem as

$$\begin{aligned} & \underset{\mathbf{r}_O}{\text{maximize}} \sum_{i \in \mathcal{M}_1} \left\{ a_i - \frac{Y_i(t) g[l_i(\mu^*)]}{h_i l_i(\mu^*)} \right\} r_{i,O} \\ & \text{subject to } \sum_{i \in \mathcal{M}_1} \frac{r_{i,O}}{l_i(\mu^*)} \leq 1, \quad r_{i,O} \leq Q_i(t), \forall i \in \mathcal{M}_1. \end{aligned} \quad (43)$$

Algorithm 2 Primal Dual Algorithm for Optimal Resource Allocation of (P4)

input : $\mathbf{x}^t, \boldsymbol{\xi}^t = \{Y_i(t), Q_i(t), A_i^t\}_{i=1}^N$
1 initialization: $\sigma_0 \leftarrow 0.1, LB \leftarrow 0, UB \leftarrow$ sufficiently large value, convert $\mathbf{x}^t$ into $\{\mathcal{M}_0, \mathcal{M}_1\}$ in (P4);
2 for each WD $j \in \mathcal{M}_0$ **do**
3 | Calculate f_j^* using (31);
4 end
5 repeat
6 | $\mu \leftarrow \frac{UB+LB}{2}$;
7 for each WD $i \in \mathcal{M}_1$ **do**
8 | Calculate $l_i(\mu)$ using (40) and $r_{i,O}^*$ using (42);
9 | $\tau_i^* \leftarrow r_{i,O}^* / l_i(\mu)$;
10 end
11 if $1 - \sum_{i \in \mathcal{M}_1} \tau_i^* < 0$ **then**
12 | $LB \leftarrow \mu$;
13 else
14 | $UB \leftarrow \mu$;
15 end
16 until $|UB - LB| \leq \sigma_0$;
17 $\mu^* \leftarrow \mu$ and obtain $\hat{\mathbf{r}}_O^*$ by solving the LP in (43), then obtain $\hat{\boldsymbol{\tau}}^*$ and $\hat{\mathbf{e}}_O^*$ using (44);
18 Return an optimal solution of (P4) by combining (31) and (44).

The above problem is a simple linear programming (LP) that can be easily solved. With a bit abuse of notation, we denote the optimal solution of (43) as $\hat{\mathbf{r}}_O^* = \{r_{i,O}^*, \forall i \in \mathcal{M}_1\}$ and retrieve the optimal solution to (32) as

$$\tau_i^* = r_{i,O}^* / l_i(\mu^*), \quad e_{i,O}^* = \frac{\tau_i^* g[l_i(\mu^*)]}{h_i l_i(\mu^*)}, \quad \forall i \in \mathcal{M}_1. \quad (44)$$

Denote $\hat{\boldsymbol{\tau}}^* = \{\tau_i^*, \forall i \in \mathcal{M}_1\}$ and $\hat{\mathbf{e}}_O^* = \{e_{i,O}^*, \forall i \in \mathcal{M}_1\}$. As $\{\hat{\boldsymbol{\tau}}^*, \hat{\mathbf{e}}_O^*, \hat{\mathbf{r}}_O^*, \mu^*\}$ satisfies the KKT conditions, $\{\hat{\boldsymbol{\tau}}^*, \hat{\mathbf{e}}_O^*, \hat{\mathbf{r}}_O^*\}$ is an optimal solution to (32). By combining the optimal solutions in (31) and (44), we obtain an optimal solution of (P4). We summarize the pseudo-code of the algorithm to solve (P4) in Algorithm 2.

V. PERFORMANCE ANALYSIS

A. Computational Complexity

We first analyze the complexity of the proposed LyDROO scheme. The execution of LyDROO algorithm consists of two parts: offloading action generation (line 4-9 of Algorithm 1) and policy update (line 10-13 of Algorithm 1). In between, offloading action generation needs to be performed in every time frame, while policy update is performed infrequently (e.g., once every tens of time frames) and in parallel with task offloading and local computation. Therefore, we focus on analyzing the complexity of generating an offloading action in each time frame. A close observation shows that the major complexity is on optimizing the resource allocation in line 7 of Algorithm 1, which executes Algorithm 2 to solve (P4) M_t times in each time frame.

We show that the time complexity of Algorithm 2 is $O\left(N \log_2\left(\frac{\Delta}{\sigma_0}\right) + N^3 \bar{L}\right)$: the first term corresponds to the

bi-section search of μ with σ_0 being the small positive precision parameter; the second term corresponds to solving the LP in (43) using interior point method [32] with $\bar{L}$ being the length of input in binary representation to problem (43). Compared to directly solving a general convex optimization (P4) with $4N$ variables using the interior point method, the proposed Algorithm 2 solves an LP in (43) with only N variables, which incurs much lower computational complexity especially when N is large. Since LyDROO executes Algorithm 2 for M_t times in each time frame, the overall complexity of generating an offloading action is $O\left(\left[N \log_2\left(\frac{\Delta}{\sigma_0}\right) + N^3 \bar{L}\right] M_t\right)$. Thanks to the adaptive procedure in (8) that gradually reduces the value of M_t during the learning process, we observe in simulations that a small M_t (e.g., less than 5 when $N = 30$) suffices to generate optimal offloading action when the learning process converges. In the Section VI, we show in simulations that the proposed LyDROO enjoys very low computation time and is suitable for online implementation in time-varying edge environment.

B. Convergence Performance

We then analyze the asymptotic convergence performance of the LyDROO algorithm in solving (P1). To begin with, we first introduce some preliminaries of Lyapunov optimization. We denote the *random event* of the considered problem as an i.i.d. process $\omega(t)$, which consists of the fading channels and data arrivals, i.e., $\omega(t) = \{h_i^t, A_i^t\}_{i=1}^N$. We introduce a class of stationary and randomized policies called ω -only policy, which observes $\omega(t)$ for each time frame t and makes control decisions independent of the queue backlog $\mathbf{Z}(t)$. To ensure that the data queue stability constraint can be satisfied, we assume (P1) is feasible and the following Slater condition holds.

Assumption 1 (Slater Condition): There are values $\epsilon > 0$ and $\Phi(\epsilon) \leq R^{opt}$ and a ω -only policy Π that makes control decisions $\alpha^{\Pi,t}$ in the t th time frame, which satisfy

$$\begin{aligned} \mathbb{E}[R^t(\alpha^{\Pi,t})] &= \Phi(\epsilon), \\ \mathbb{E}[e_i^t(\alpha^{\Pi,t})] &\leq \gamma_i - \epsilon, \quad \forall i. \\ \mathbb{E}[A_i^t] &\leq \mathbb{E}[D_i^t(\alpha^{\Pi,t})] - \epsilon, \quad \forall i. \end{aligned} \quad (45)$$

Here, $R^t \triangleq \sum_{i=1}^N c_i r_i^t$ denotes the weighted sum computation rate archived in the t th time frame. R^{opt} is the optimal objective of (P1) obtained over all feasible control policies (including but not limited to ω -only policy).

We show the performance of LyDROO algorithm in the following Theorem 1.

Theorem 1: Suppose that (P1) is feasible and satisfies the Slater condition for some ϵ , $\Phi(\epsilon)$ and ω -only policy Π . Suppose that given any $\mathbf{Z}(t)$ in time frame t , the LyDROO algorithm produces a value of (19) that is no larger than a constant $C \geq 0$ above the minimum, i.e., the per-frame subproblem (P2) is solved within an optimality gap C . Then, the following conditions hold when applying the LyDROO algorithm in each time frame t

- a) The time average computation rate satisfies

$$\lim_{K \rightarrow \infty} 1/K \cdot \sum_{t=1}^K \mathbb{E}[R^t] \geq R^{opt} - (\hat{B} + C)/V. \quad (46)$$

- b) The average system queue length satisfies

$$\begin{aligned} \lim_{K \rightarrow \infty} 1/K \cdot \sum_{t=1}^K \sum_{i=1}^N \mathbb{E}[Q_i^t] \\ \leq 1/\epsilon \cdot (\hat{B} + C + V [R^{opt} - \Phi(\epsilon)]). \end{aligned} \quad (47)$$

- c) All the data queues $Q_i(t)$ are strongly stable and the time average power constraint (6c) is satisfied with probability 1.

Proof: Please see the detailed proof in the Appendix B. ■

Theorem 1 indicates that if the LyDROO algorithm achieves a limited optimality gap C when solving the per-frame subproblem (P2), then we satisfy all the long-term constraints and achieve an $[O(1/V), O(V)]$ computation rate-delay tradeoff. That is, by increasing V , we can improve the objective of (P1) proportional to $1/V$, but at the cost of longer data queue length (processing delay) proportional to V , and vice versa. Besides, a smaller C leads to both higher rate and lower delay performance. In simulation section, we demonstrate the impact of V on the long-term performance and show that LyDROO achieves very small C for the per-frame subproblem. Notice that the above analysis does not assume the specific utility function in the objective of (P1), and thus the results hold for any non-decreasing concave utility function $U(r_i^t)$.

VI. SIMULATION RESULTS

In this section, we use simulations to evaluate the performance of the proposed LyDROO algorithm. All the computations are evaluated on a TensorFlow 2.0 platform with an Intel Core i5-4570 3.2GHz CPU and 12 GB of memory. We assume that the average channel gain $\bar{h}_i$ follows a path-loss model $\bar{h}_i = A_d \left(\frac{3 \times 10^8}{4\pi f_c d_i}\right)^{d_e}$, $i = 1, \dots, N$, where $A_d = 3$ denotes the antenna gain, $f_c = 915$ MHz denotes the carrier frequency, $d_e = 3$ denotes the path loss exponent, and d_i in meters denotes the distance between the i th WD and the ES. h_i follows an i.i.d. Rician distribution with line-of-sight link gain equal to $0.3\bar{h}_i$. The noise power $N_0 = Wv_0$ with noise power spectral density $v_0 = -174$ dBm/Hz. Unless otherwise stated, we consider $N = 10$ WDs equally spaced with $d_i = 120 + 15(i - 1)$, for $i = 1, \dots, N$. The weight $c_i = 1.5$ if i is an odd number and $c_i = 1$ otherwise. The task data arrivals of all the WDs follow exponential distribution with equal average rate $\mathbb{E}[A_i^t] = \lambda_i$, $i = 1, \dots, N$. The values of the other parameters are listed in Table I, which are equal for all the WDs.

The proposed LyDROO adopts a fully-connected multilayer perceptron in the actor module, consisting of one input layer, two hidden layers, and one output layer, where the first and second hidden layers have 120 and 80 hidden neurons,

TABLE I
SIMULATION PARAMETERS

$W = 2$ MHz	$f_i^{max} = 0.3$ GHz	$P_i^{max} = 0.1$ watt
$v_u = 1.1$	$\kappa = 10^{-26}$	$\phi = 100$
$q = 1024$	$\delta_T = 10$	$\delta_M = 32$
$\gamma_i = 0.08$ watt	$\nu = 1000$	$ S^t = 32$
$V = 20$	$\lambda_i = 3$ Mbps	$T = 1$ second

respectively. For performance comparison, we consider two benchmark methods:

- Lyapunov-guided Coordinate Decent (LyCD): It minimizes the upper bound of drift-plus-penalty in (19), or equivalently solves (P2), using the coordinate decent (CD) method [8] that iteratively applies one-dimensional search to update the binary offloading decision vector $\mathbf{x}^t$. Although the optimal solution of (P2) is hard to obtain, we have verified through extensive simulations that the CD method achieves close-to-optimal performance. Therefore, we consider LyCD as a target performance benchmark of the LyDROO algorithm. The major drawback of LyCD, however, lies in the significant computation delay when N is large. We show in the following simulations that the proposed LyDROO achieves the similar computation performance as LyCD but takes much shorter computation time.
- Myopic optimization [18]: The Myopic method neglects the data queue backlogs and maximizes the weighted sum computation rate in each time frame t by solving

$$\underset{\mathbf{x}^t, \tau^t, \mathbf{f}^t, \mathbf{e}_O^t, \mathbf{r}_O^t}{\text{maximize}} \quad \sum_{i=1}^N c_i r_i^t \quad (48a)$$

$$\text{subject to } (21a) - (21e), \quad (48b)$$

$$e_i^t \leq t\gamma_i - \sum_{l=1}^{t-1} e_i^l, \quad \forall i. \quad (48c)$$

Here, constraint (48c) guarantees that the i th average power constraint of (P1) is satisfied up to the t th time frame, where $\{e_i^l | l < t\}$ is the past energy consumptions known at the t th time frame.

Besides the two benchmarks above, we have also considered using Deep Deterministic Policy Gradient (DDPG) [33], a state-of-the-art policy-based DRL scheme, to directly learn the optimal mapping from the input ξ^t to the output mixed integer-continuous offloading action $\{x_i^t, \tau_i^t, f_i^t, e_{i,O}^t\}_{i=1}^N$ in (P2). However, we find through extensive simulations that DDPG is unable to stabilize the task data queues for all the WDs, even when the number of WDs and task arrival rates are small, e.g., $N = 3$ and $\lambda_i = 3$ Mbps. Therefore, we do not include DDPG as a performance benchmark in the following simulations.

In Fig. 4, we first evaluate the performance of the LyDROO algorithm in solving per-frame subproblem (P2). For fair comparison, we first apply the LyCD method for 30,000 time frames, where we record the input to the actor module $\{\xi(t)\}$ throughout the time. Then, we use the same $\{\xi(t)\}$ as the input to the LyDROO framework in Fig. 3 only for computing the output action $\{\mathbf{x}^t, \mathbf{y}^t\}$ in each time frame without updating the queue states. We plot the ratio between the objective values of (P2) achieved by the LyDROO and LyCD as the time proceeds, where each point is a moving-window average of 500 time frames. We notice that the ratio gradually increases with time and eventually reaches about 0.96. We also show the boxplot of the last 500 time frames, which shows that the medium ratio is around 0.98 and the ratio is above 0.94 for more than 75% of the cases. As LyCD achieves close-to-optimal performance of the per-frame subproblem (P2), this shows that LyDROO solves (P2) with small optimality gap C , thus leading to both

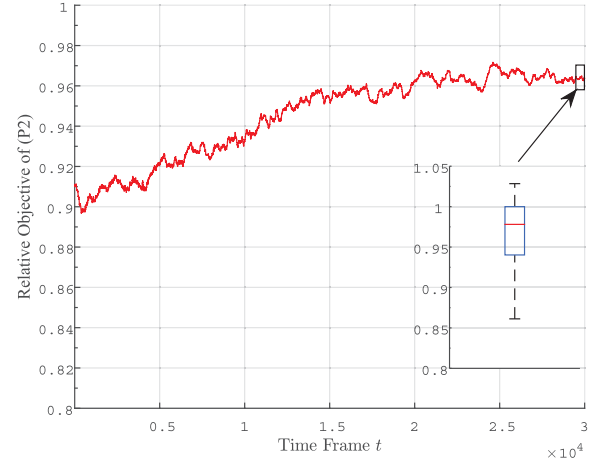


Fig. 4. Performance of the LyDROO algorithm in solving per-frame subproblem (P2). In the boxplot, the central mark (in red) indicates the median, and the bottom and top edges of the box indicate the 25th and 75th percentiles, respectively.

higher computation rate and lower execution delay according to Theorem 1.

We then evaluate the convergence of proposed LyDROO algorithm and the two benchmark methods. In Fig. 5, we consider two data arrival rates with $\lambda_i = 2.5$ and 3 Mbps for all i , and plot the weighted sum computation rate, average data queue length, and average power consumption performance over time. We consider i.i.d. realizations of random events $\omega(t)$ in 10,000 time frames, where each point in the figure is a moving-window average of 200 time frames. In Fig. 5(a), we observe that for a low data arrival rate $\lambda_i = 2.5$, all the schemes maintain the data queues stable and achieve similar computation rate performance. Besides, they all satisfy the average power constraint 0.08 W in Fig. 5(b). In particular, the LyDROO and LyCD methods achieve higher data queue lengths than the Myopic scheme, as they consume strictly lower power than the average power requirement, meanwhile achieving the identical rate performance in Fig. 5(c). When we increase λ_i from 2.5 to 3, all the three schemes still satisfy the average power constraints. However, the average data queue length of the Myopic method increases almost linearly with time, indicating that it is unable to stabilize the data queues. This is because the data arrival rate has surpassed the computation capacity (i.e., achievable sum computation rate) of the Myopic algorithm. On the other hand, both the LyCD and LyDROO methods can stabilize the data queues, indicating that the proposed Lyapunov-based method can achieve a higher computation capacity than the Myopic method. In between, the LyCD method maintains lower data queue length over all time frames. The LyDROO method takes time to learn the optimal offloading policy in the early stage, where the data queue length increases quickly when $t \leq 3,000$. However, as the embedded DNN gradually approaches the optimal policy, the data queue length quickly drops and eventually converges to the similar queue length and rate performance as the LyCD method after around $t = 7,500$. For both λ_i 's, the data queue lengths of the LyDROO

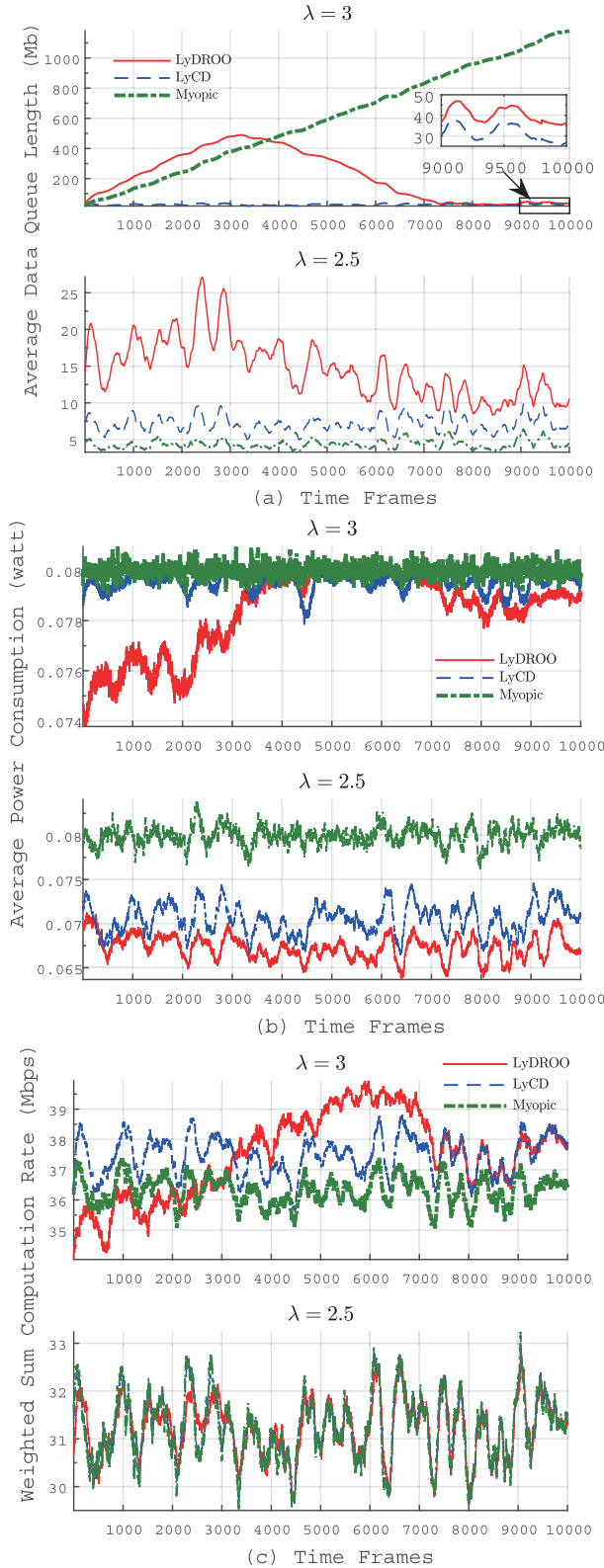


Fig. 5. Convergence performance comparisons of different schemes under $\lambda_i = 2.5$ and 3. From top to bottom figures: a) data queue length; b) power consumption; c) weighted sum computation rate.

algorithm start to drop at around $t = 3,000$, indicating its fast convergence even under highly dynamic queueing systems. We also notice that the proposed LyDROO achieves excellent computation performance even before the learning algorithm

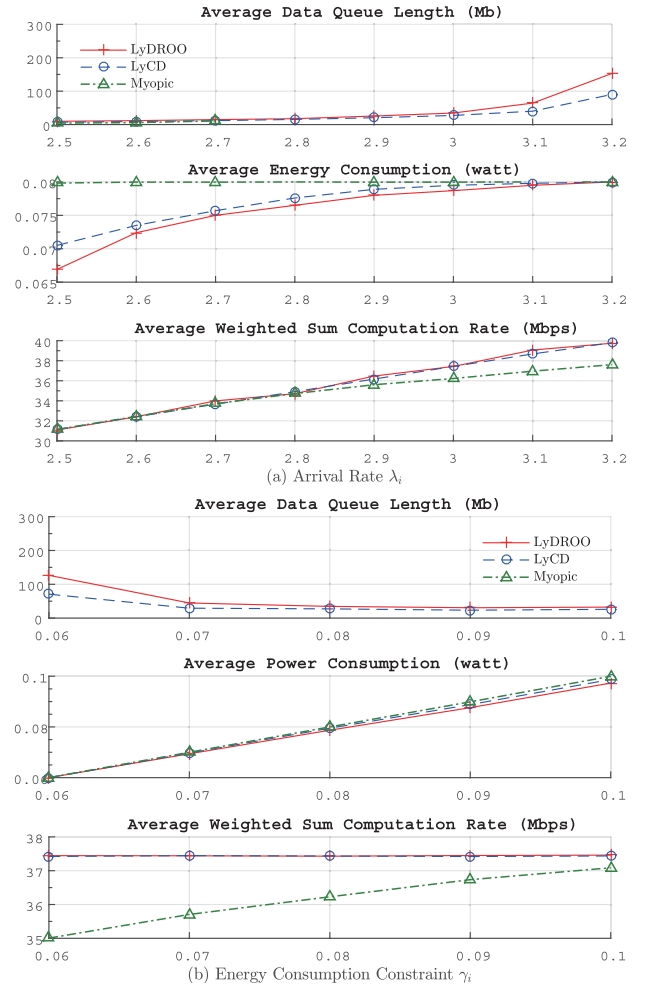


Fig. 6. Performance comparisons under different λ_i and γ_i .

fully converges. In Fig. 5(c) with $\lambda_i = 3$ Mbps, the performance gap of computation rate is less than 5% when $t \leq 3000$ compared to the target benchmark LyCD, and LyDROO even achieves higher rate than LyCD between $t = 3000$ to 7000 when the learning process gradually converges.

In Fig. 6, we evaluate the impact of system parameters. In Fig. 6(a), we fix $\gamma_i = 0.08$ watt and vary data arrival rate λ_i from 2.5 to 3.2 Mbps. In Fig. 6(b), we fix $\lambda_i = 3$ and vary power constraint γ_i from 0.06 to 0.1. We omit the results for $\lambda_i \geq 3.3$ and $\gamma_i \leq 0.05$ in the two figures, respectively, because we observe that none of the three schemes can maintain queue stability under the heavy data arrivals and stringent power constraints, i.e., arrival rate surpasses the achievable sum computation rate. All the three schemes satisfy the average power constraints under different parameters in both figures. In Fig. 6(a), the data queue lengths of all the three schemes increase with λ_i . In particular, the data queues are stable with LyCD and LyDROO under all the considered λ_i , while the queue lengths of the Myopic scheme become infinite when $\lambda_i \geq 2.8$. In Fig. 6(b), the data queues are stable with LyCD and LyDROO under all the considered λ_i , and the queue length decreases with γ_i under the less stringent power constraint. In vivid contrast, the Myopic scheme has infinite queue length under all λ_i (thus, no point appears in the queue

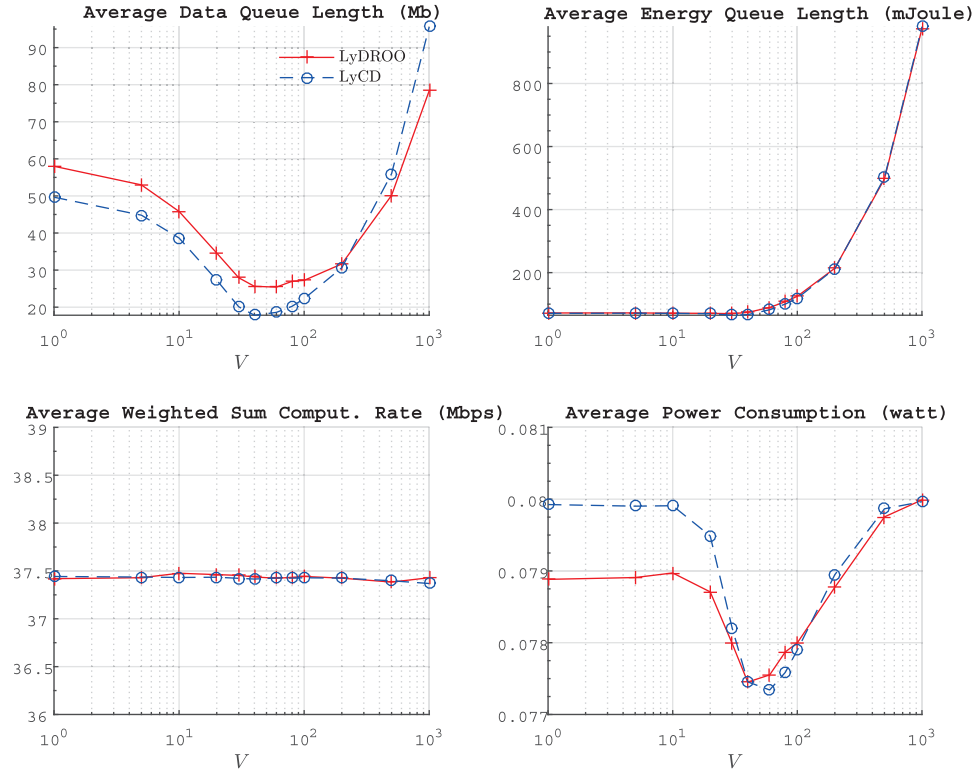


Fig. 7. Impact of the Lyapunov control parameter V .

length figure). The results show that both LyDROO and LyCD achieve much larger stable capacity region than the Myopic method, and thus are more robust under heavy workload and stringent power constraints. We also observe that LyCD and LyDROO achieve identical computation rate performance in all the considered cases. This is because when the data queues are long-term stable, the average computation rate of the i th WD (departures rate of the data queue) equals the data arrival rate λ_i , and thus the achievable average weighted sum computation rate is $\sum_{i=1}^N c_i \lambda_i$ for both schemes. In fact, this also indicates that both LyDROO and LyCD achieve the optimal computation rate performance in all the considered setups. In contrast, the Myopic method achieves lower computation rate when the data queues are unstable, i.e., for $\lambda_i > 2.7$ in Fig. 6(a) and all the considered $\gamma_i \in [0.06, 0.1]$ in Fig. 6(b). Moreover, the performance gap increases under heavier workload (larger λ_i) and more stringent power constraints (smaller γ_i).

In Fig. 7, we further show the impact of the Lyapunov control parameter V on the performance of the two Lyapunov-based LyDROO and LyCD methods, where $V \in [1, 1000]$. All the points in the figure are the average performance after convergence. In all the figures, the two methods achieve very similar performance, where they both maintain data and energy queues stable, control the average power consumption strictly below the threshold, and achieve the optimal computation rate performance. The parameter V controls the balance between the sum computation rate performance and total data queue length. Interestingly, when V is small (e.g., $V \leq 40$), the data queue length and power consumption

decrease with the increase of V , and the virtual energy queue length is close to zero. This is because the offloading probabilities increase for most of WDs as V becomes larger. However, when $V > 40$, the data queue length, power consumption, and energy queue length all increase with V monotonically. This is because the offloading strategy becomes unfair when V is large, such that the increase of offloading probabilities of some WDs is at the cost of decreased offloading probabilities of many others. This results in an overall increase of average data queue length and energy consumption. In practice, we should set a moderate V to reduce the task data buffer size required at the WDs, which depends on the specific network deployment and the task arrival rates of all the WDs.

In Fig. 8, we show the performance of LyDROO under different number of WDs. Specifically, we plot in Fig. 8(a) the average queue length when the individual task arrival rate λ_i varies. We observe that LyDROO can maintain stable task data queue for $\lambda_i \leq 3.2$ Mbps when $N = 10$, $\lambda_i \leq 2.4$ Mbps when $N = 20$, and $\lambda_i \leq 2$ Mbps when $N = 30$. The points where task data queue becomes unstable are not plotted, e.g., $\lambda_i \geq 2.5$ Mbps for $N = 20$. As expected, the stable capacity region shrinks with N because of the heavier computation workload in the system under the same λ_i . For a specific individual task arrival rate λ_i , the average data queue length increases with N . For instance, for $\lambda_i = 2$ Mbps, the queue length is less than 5 when $N = 10$, around 20 when $N = 20$, and around 50 when $N = 30$. In Fig. 8(b), we observe that the energy consumption increases with λ_i for all N , and gradually reaches the power consumption threshold 0.08 Watt when λ_i approaches the upper boundary of stable capacity region. The higher

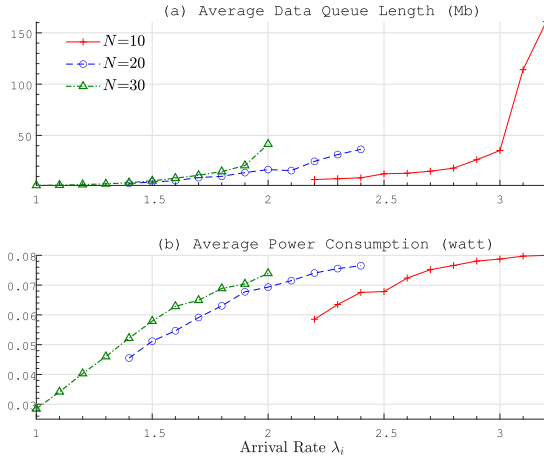


Fig. 8. Performance of LyDROO under different number of WDs $N \in \{10, 20, 30\}$.

TABLE II
COMPUTATION RATE AND CPU COMPUTATION TIME WHEN N VARIES

N	Computation rate (Mbps)		CPU computation time (second)		
	LyDROO	LyCD	LyDROO	LyCD	LyCD LyDROO
10	37.43	37.43	0.021	0.27	12.86
20	37.61	37.60	0.108	2.57	23.80
30	37.36	37.36	0.156	8.02	51.41

power consumption arises from the more stringent resource constraint when the overall network computation workload increases.

From the above discussions, both LyDROO and LyCD achieve excellent computation performance under different parameters. In Table II, we compare their computation time under different number of WDs N . Here, we consider a fixed total network workload 30 Mbps and equally allocate $\lambda_i = 30/N$ to each WD for $N \in \{10, 20, 30\}$. The locations of the N WDs are evenly spaced within [120, 255] meters distance to the ES. We observe that the two methods achieve similar computation rate performance for all N and all the long-term constraints are satisfied. Besides, thanks to setting a time-varying M_t in (8), LyDROO achieves significant saving in execution time compared to that when a fixed $M_t = 2N$ is used, e.g., saves more than 80% execution time for $N = 30$, without degrading the convergence. Due to the page limit, we omit the illustrations of detailed performance and focus on comparing the computation time between LyCD and LyDROO methods. In Table II, LyDROO takes at most 0.156 second to generate an offloading action in all the cases. In contrast, LyCD consumes acceptable latency when $N = 10$, but significantly long latency when $N = 30$, e.g., around 50 times longer than that of LyDROO method. Because the channel coherence time of a common indoor IoT system is no larger than several seconds, the long computation time makes LyCD costly even infeasible in a practical MEC system with online offloading decision. The proposed LyDROO algorithm, in contrast, incurs very short latency overhead, e.g., around 3% overhead when the time frame is 5 seconds for $N = 30$. Recall that after the DNN generating a control action in a time

frame, the training process of the DNN is performed in parallel with task offloading and computation in the remainder of the time frame, and thus does not incur additional delay overhead. Therefore, the LyDROO algorithm can be efficiently applied in an MEC system under fast channel variation.

VII. CONCLUSION AND DISCUSSIONS

In this paper, we have studied an online stable computation offloading problem in a multi-user MEC network under stochastic wireless channel and task data arrivals. We formulate a multi-stage stochastic MINLP problem that maximizes the average weighted sum computation rate of all the WDs under long-term queue stability and average power constraints. The online design requires making joint action of binary computation offloading and resource allocation in each short time frame without the assumption of knowing the future realizations of random channel conditions and data arrivals. To tackle the problem, we proposed a LyDROO framework that combines the advantages of Lyapunov optimization and DRL. We show in both theory and simulations that the proposed approach can achieve optimal computation rate performance meanwhile satisfying all the long-term constraints. Besides, it incurs very low computational complexity in generating an online action, and converges within relatively small number of iterations. The proposed LyDROO framework has wide application in MEC networks in enhancing both the efficiency and robustness of computation performance.

We conclude the paper with some potential extensions of the proposed LyDROO scheme and future working directions. First, besides binary computation offloading considered in this paper, the proposed LyDROO scheme can also be extended to design online partial computation offloading strategy where the computation tasks consist of multiple independent subtasks (such as in [19]). By carefully setting binary variables to represent which subset of subtasks to be offloaded for edge execution, LyDROO is applicable to jointly optimize the binary offloading decisions and continuous resource allocation for the partial offloading scheme.

Second, we consider in this paper that the task data arrivals follow an i.i.d. process, which is a crucial assumption for the proof of the convergence performance of the LyDROO scheme in Algorithm 1. However, according to Theorem 4.9 in [21], the proposed LyDROO can achieve the similar $[O(1/V), O(V)]$ performance guarantees as those described in Theorem 1 of this paper when the task data arrivals follow a more general ergodic (possibly non-i.i.d.) process, such as a Markov modulated process that the distribution of arrival rates is time-varying and correlated in time. In Fig. 9, we evaluate the performance of LyDROO under non-i.i.d. task arrivals for $N = 10$ WDs, where the task arrivals follow an ON-OFF Markov modulated random process. Specifically, we consider two states for the arrival process, i.e., the ON state and the OFF state, which are modulated by a two-state Markov chain with transition matrix $[0.1, 0.9; 0.9, 0.1]$. The arrived task data size A_i^t at the i th WD in the t th time frame is 0 if the system is in OFF state, and follows an i.i.d. exponential distribution in ON state. In practice, the ON-OFF Markov modulated random process models

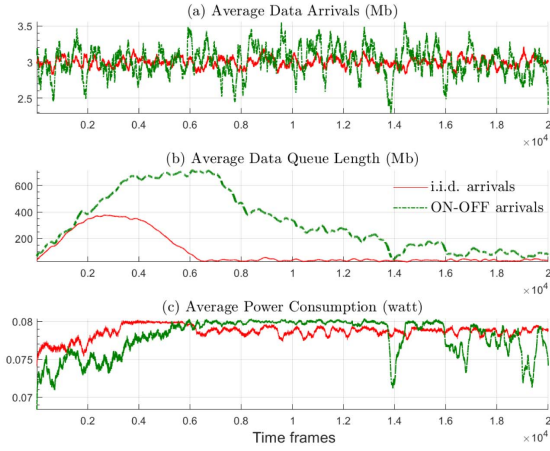


Fig. 9. The convergence performance of LyDROO under both i.i.d. exponential and the non-i.i.d. Markov modulated ON-OFF task arrival models.

the bursty arrivals of task data. We compare the convergence performance of LyDROO under both i.i.d. exponential and the non-i.i.d. ON-OFF task arrival models. For fair comparison, we set equal long-term average task arrivals rate 3 Mbps for both data arrival models. We plot in Fig. 9(a) the average task arrival of the 10 WDs over different time frames of both i.i.d. and the non-i.i.d. ON-OFF task arrival models. We observe in Fig. 9(b) that LyDROO can achieve stable task data queue, and in fact very low task queue length, for both i.i.d. and non-i.i.d. task data arrivals after sufficient training, although the time until convergence is longer under the non-i.i.d. arrivals. In Fig. 9(c), the average energy consumption constraint 0.08 watt is also satisfied under both task arrival models. The results demonstrate the effectiveness of the proposed LyDROO under non-i.i.d. task data arrivals.

Third, we assume a block fading channel model in this paper. In practice, however, wireless channel may experience small variations within a time frame. Recall that h_i^t denotes the channel gain at the beginning of the t th time frame. In case of small channel variation, we can include a signal-to-noise (SNR) power margin $\rho \geq 1$ when setting the computation offloading rate, i.e., $D_{i,O}^t = \frac{W\tau_i^t T}{v_u} \log_2 \left(1 + \frac{E_{i,O}^t h_i^t}{\tau_i^t T \rho N_0} \right)$ in (2), such that the channel gain is likely above h_i^t / ρ throughout the time frame. Evidently, setting a larger ρ increases the robustness of communication against channel variation, however, at the cost of lower spectrum efficiency.

Fourth, we neglect in this paper the delay on downloading the computation result from the edge server. When the downloading time is non-negligible for some application, we denote the delay on downloading the result of the i th offloading WD in the t th time frame as

$$w_i^t = \frac{L_i v_u}{W \log_2 \left(1 + \frac{P_0 g_i^t}{N_0} \right)}, \quad \forall i \in \mathcal{M}_1^t, \quad (49)$$

where g_i^t denotes the downlink channel gain, P_0 denotes the fixed transmit power of the edge base station, and L_i denotes the fixed size of computation result. During the execution of the LyDROO algorithm, $\mathcal{M}_1^t$ is the output of the actor module, such that w_i^t 's are fixed parameters when the critic

module solves the optimal resource allocation problem (P4) given $\mathcal{M}_1^t$. Therefore, we can include result downloading delay into consideration by simply replacing the time allocation constraint in (P4) $\sum_{i \in \mathcal{M}_1^t} \tau_i \leq 1$ with the similar linear constraint $\sum_{i \in \mathcal{M}_1^t} (\tau_i^t + w_i^t) \leq 1$, without affecting the overall algorithm design of LyDROO.

Last but not the least, in this paper, we coordinate the computation offloading of multiple WDs using TDMA. In fact, the proposed LyDROO is also applicable to MEC systems using other multiple access methods, such as FDMA, CDMA, OFDMA, and NOMA (non-orthogonal multiple access), as long as the critic module can quickly obtain the optimal wireless resource allocation. Accordingly, the technical challenge lies in the design of efficient resource allocation algorithms under different multiple access schemes considered.

APPENDIX I PROOF OF PROPOSITION 1

Proof: Given $r_{i,O}$, we denote the objective of the problem (39) as $\Omega(\tau_i)$, which is a strictly concave function within the feasible set $\tau_i \geq \frac{r_{i,O}}{R_i^{max}}$. Accordingly, the minimum is achieved at either the boundary point $\frac{r_{i,O}}{R_i^{max}}$ or the point v_1 that satisfies $\Omega'(v_1) = 0$, depending on the value of v_1 . To obtain v_1 , we take the derivative of $\Omega(\tau_i)$ and set it equal to zero, i.e.,

$$\begin{aligned} \Omega'(\tau_i) &= -\mu - \frac{Y_i(t)N_0}{h_i} \\ &\quad \left(2^{\frac{r_{i,O}v_u}{W\tau_i}} - 1 - \ln 2 \cdot 2^{\frac{r_{i,O}v_u}{W\tau_i}} \cdot \frac{r_{i,O}v_u}{W\tau_i} \right) \\ &= -\frac{Y_i(t)N_0 e}{h_i} \left[e^{-1} \left(\frac{\mu h_i}{Y_i(t)N_0} - 1 \right) \right. \\ &\quad \left. - e^{\ln 2 \cdot \frac{r_{i,O}v_u}{W\tau_i} - 1} \left(\ln 2 \cdot \frac{r_{i,O}v_u}{W\tau_i} - 1 \right) \right] = 0, \\ &\Rightarrow e^{\ln 2 \cdot \frac{r_{i,O}v_u}{W\tau_i} - 1} \left(\ln 2 \cdot \frac{r_{i,O}v_u}{W\tau_i} - 1 \right) \\ &= e^{-1} \left(\frac{\mu h_i}{Y_i(t)N_0} - 1 \right). \end{aligned} \quad (50)$$

Because $e^{-1} \left(\frac{\mu h_i}{Y_i(t)N_0} - 1 \right) \geq -1$, the above equality is equivalent to

$$\ln 2 \cdot \frac{r_{i,O}v_u}{W\tau_i} - 1 = \mathcal{W} \left(e^{-1} \left[\frac{\mu h_i}{Y_i(t)N_0} - 1 \right] \right), \quad (51)$$

where $\mathcal{W}(x)$ denotes the Lambert-W function. Therefore, we have

$$v_1 = \frac{\ln 2 \cdot v_u \cdot r_{i,O}}{W \cdot \left[\mathcal{W} \left(e^{-1} \left[\frac{\mu h_i}{Y_i(t)N_0} - 1 \right] \right) + 1 \right]}. \quad (52)$$

If $v_1 < \frac{r_{i,O}}{R_i^{max}}$, or equivalently $\Omega'(\tau_i) = 0$ is not achievable within the feasible set, we can infer that the optimal solution is obtained at the boundary $(\tau_i)^* = \frac{r_{i,O}}{R_i^{max}}$. Because $\Omega(\tau_i)$ is concave, $\Omega'(\tau_i)$ is a decreasing function. Given $\Omega'(v_1) = 0$, the condition $v_1 < \frac{r_{i,O}}{R_i^{max}}$ is equivalent to $\Omega' \left(\frac{r_{i,O}}{R_i^{max}} \right) < 0$.

By substituting $\tau_i = \frac{r_{i,O}}{R_i^{max}}$ into (50), we have $v_1 < \frac{r_{i,O}}{R_i^{max}}$ when

$$\begin{aligned} \mu + Y_i(t)P_i^{max} \left[1 - \ln(1 + d_i) \left(\frac{1}{d_i} + 1 \right) \right] &> 0 \\ \Rightarrow \ln(1 + d_i) &\leq \left(1 + \frac{\mu}{Y_i(t)P_i^{max}} \right) \left(1 - \frac{1}{1 + d_i} \right) \\ \Rightarrow \ln \left(\frac{1}{1 + d_i} \right) &\geq -A_i + \frac{A_i}{1 + d_i}, \end{aligned} \quad (53)$$

where $d_i \triangleq \frac{h_i P_i^{max}}{N_0}$ and $A_i \triangleq 1 + \frac{\mu}{Y_i(t)P_i^{max}}$. By taking a natural exponential operation at both sides of (53), we have

$$\begin{aligned} \exp \left(-\frac{A_i}{1 + d_i} \right) \left(\frac{1}{1 + d_i} \right) &\geq \exp(-A_i) \\ \Rightarrow \exp \left(-\frac{A_i}{1 + d_i} \right) \left(-\frac{A_i}{1 + d_i} \right) &\leq -A_i \exp(-A_i). \end{aligned}$$

Because the RHS of the above inequality satisfies $-e^{-1} \leq -A_i \exp(-A_i) \leq 0$, the inequality can be equivalently expressed as

$$-A_i/(1 + d_i) \leq \mathcal{W}(-A_i \exp(-A_i)), \quad (54)$$

where $\mathcal{W}(-A_i \exp(-A_i)) \in [-1, 0]$. The equivalence holds because $\mathcal{W}(x)$ is an increasing function when $x \geq -1/e$. After some simple manipulation, we obtain from (54) that the optimal solution $(\tau_i)^* = \frac{r_{i,O}}{R_i^{max}}$ when $h_i \leq \frac{N_0}{P_i^{max}} \left(\frac{A_i}{-\mathcal{W}(-A_i \exp(-A_i))} - 1 \right)$. Otherwise, we conclude that $v_1 \geq \frac{r_{i,O}}{R_i^{max}}$ and $\Omega'(\tau_i) = 0$ is achievable such that the optimal solution is $\tau_i^* = v_1$. ■

APPENDIX II PROOF OF THEOREM 1

To prove Theorem 1, we first introduce the following two lemmas.

Lemma 1: Suppose that (P1) is feasible and $\omega(t)$ is stationary, then for any $\delta > 0$, there exists an ω -only policy Γ , such that the following inequalities are satisfied:

$$\begin{aligned} \mathbb{E}[R^t(\alpha^{\Gamma,t})] &\geq R^{opt} - \delta, \\ \mathbb{E}[e_i^t(\alpha^{\Gamma,t}) - \gamma_i] &\leq \delta, \quad \forall i, \\ \mathbb{E}[A_i^t] &\leq \mathbb{E}[D_i^t(\alpha^{\Gamma,t})] + \delta, \quad \forall i. \end{aligned} \quad (55)$$

Proof: See Theorem 4.5 of [21] for detailed proof. ■

Lemma 2: If $Y_i(t)$ is rate stable, i.e., $\lim_{K \rightarrow \infty} \frac{Y_i(K)}{K} = 0$ holds with probability 1, then the i th average power constraint in (6c) is satisfied with probability 1.

Proof: Using the sample path property (Lemma 2.1 of [21]), we have

$$\begin{aligned} \frac{Y_i(K)}{K} - \frac{Y_i(1)}{K} &\geq \frac{1}{K} \sum_{t=1}^K e_i^t - \frac{1}{K} \sum_{t=1}^K \gamma_i \\ &\Rightarrow \frac{1}{K} \sum_{t=1}^K e_i^t \leq \gamma_i + \frac{Y_i(K)}{K}. \end{aligned} \quad (56)$$

By taking the limit $K \rightarrow \infty$ on both sides and substituting $\lim_{K \rightarrow \infty} \frac{Y_i(K)}{K} = 0$, we have $\lim_{K \rightarrow \infty} \frac{1}{K} \sum_{t=1}^K e_i^t \leq \gamma_i$ holds with probability 1, which completes the proof. ■

Proof of Theorem 1: Because (P1) is feasible and $\omega(t)$ is an i.i.d. process, we apply Lemma 1 and consider a fixed $\delta > 0$

and the corresponding ω -only control policy Γ . Because the minimum of (19) is obtained over all feasible control policies, including Γ , we have

$$\begin{aligned} \Delta L(\mathbf{Z}(t)) - V \cdot \mathbb{E}[R^t | \mathbf{Z}(t)] &\leq \hat{B} + C + \sum_{i=1}^N \left(Q_i(t) \mathbb{E}[(A_i^t - D_i^t(\alpha^{\Gamma,t})) | \mathbf{Z}(t)] \right. \\ &\quad \left. + Y_i(t) \mathbb{E}[e_i^t(\alpha^{\Gamma,t}) - \gamma_i | \mathbf{Z}(t)] - V \cdot \mathbb{E}[R^t(\alpha^{\Gamma,t}) | \mathbf{Z}(t)] \right) \\ &\stackrel{\dagger}{\leq} \hat{B} + C + \sum_{i=1}^N Q_i(t) \mathbb{E}[(A_i^t - D_i^t(\alpha^{\Gamma,t}))] \\ &\quad + \sum_{i=1}^N Y_i(t) \mathbb{E}[e_i^t(\alpha^{\Gamma,t}) - \gamma_i] - V \cdot \sum_{i=1}^N \mathbb{E}[R^t(\alpha^{\Gamma,t})] \\ &\stackrel{\ddagger}{\leq} \hat{B} + C + \delta \left[\sum_{i=1}^N (Q_i(t) + Y_i(t)) \right] - V(R^{opt} - \delta), \end{aligned} \quad (57)$$

where inequality ($\dagger$) is because the control policy Γ is independent to queue backlog $\mathbf{Z}(t)$, and the inequality ($\ddagger$) is obtained by plugging (55). By letting $\delta \rightarrow 0$, we have

$$\Delta L(\mathbf{Z}(t)) - V \cdot \mathbb{E}[R^t | \mathbf{Z}(t)] \leq \hat{B} + C - V R^{opt}. \quad (58)$$

Furthermore, by summing both sides of (58) from $t = 1$ to K , and taking iterated expectations and telescoping sums, then dividing both sides by KV , we obtain

$$\begin{aligned} \frac{1}{KV} \left(\mathbb{E}[L(\mathbf{Z}(K+1))] - \mathbb{E}[L(\mathbf{Z}(1))] - \sum_{t=1}^K \mathbb{E}[R^t] \right) &\leq (\hat{B} + C)/V - R^{opt}. \end{aligned} \quad (59)$$

Because $L(\mathbf{Z}(K+1)) \geq 0$ and $L(\mathbf{Z}(1)) = 0$, we prove a) by letting $K \rightarrow \infty$ in (59).

To prove b), we consider the ω -only policy Π that satisfies the Slater condition for some values ϵ and $\Phi(\epsilon)$. By plugging the policy Π to the RHS of the inequality ($\dagger$) in (57), we have

$$\begin{aligned} \Delta L(\mathbf{Z}(t)) - V \cdot \mathbb{E}[R^t | \mathbf{Z}(t)] &\leq \hat{B} + C - \epsilon \left[\sum_{i=1}^N (Q_i(t) + Y_i(t)) \right] - V\Phi(\epsilon), \end{aligned} \quad (60)$$

where the inequality is obtained from (45). Taking iterated expectations, summing the telescoping series, and rearranging terms yields

$$\begin{aligned} \frac{1}{K} \sum_{t=1}^K \sum_{i=1}^N \mathbb{E}[(Q_i(t) + Y_i(t))] &\leq \frac{\hat{B} + C + V \left(\frac{1}{K} \cdot \sum_{t=1}^K \mathbb{E}[R^t] - \Phi(\epsilon) \right)}{\epsilon} + \frac{\mathbb{E}[L(\mathbf{Z}(1))]}{\epsilon K}. \end{aligned}$$

By letting $K \rightarrow \infty$ and plugging the fact that $\lim_{K \rightarrow \infty} \frac{1}{K} \sum_{t=1}^K \mathbb{E}[R^t] \leq R^{opt}$, we have

$$\begin{aligned} \lim_{K \rightarrow \infty} \frac{1}{K} \sum_{t=0}^{K-1} \sum_{i=1}^N \mathbb{E}[(Q_i(t) + Y_i(t))] &\leq \frac{\hat{B} + C + V(R^{opt} - \Phi(\epsilon))}{\epsilon}. \end{aligned} \quad (61)$$

Then, (47) in b) is proved because $Y_i(t) \geq 0$. Meanwhile, (61) also indicates that

$$\begin{aligned} \lim_{K \rightarrow \infty} \frac{1}{K} \sum_{t=1}^K \mathbb{E}[Q_i(t)] &< \infty, \quad \forall i, \\ \lim_{K \rightarrow \infty} \frac{1}{K} \sum_{t=1}^K \mathbb{E}[Y_i(t)] &< \infty, \quad \forall i. \end{aligned} \quad (62)$$

That is, all the data queues and virtual queues are strongly stable. Because strong stability implies rate stability (Theorem 2.8 of [21]), we have $Y_i(t)$ is rate stable. By Lemma 2, the average power constraint (6c) is satisfied with probability 1, which lead to the proof of c). ■

REFERENCES

- [1] S. Bi, L. Huang, H. Wang, and Y. J. Zhang, "Stable online computation offloading via Lyapunov-guided deep reinforcement learning," in *Proc. IEEE ICC*, Montreal, QC, Canada, Jun. 2021, pp. 1–7. [Online]. Available: <https://arxiv.org/abs/2102.03286>.
- [2] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, Aug. 2017.
- [3] W. Zhang, Y. Wen, K. Guan, D. Kilper, H. Luo, and D. O. Wu, "Energy-optimal mobile cloud computing under stochastic wireless channel," *IEEE Trans. Wireless Commun.*, vol. 12, no. 9, pp. 4569–4581, Sep. 2013.
- [4] C. You, K. Huang, and H. Chae, "Energy efficient mobile cloud computing powered by wireless energy transfer," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 5, pp. 1757–1771, May 2016.
- [5] J. Yan, S. Bi, Y. J. Zhang, and M. Tao, "Optimal task offloading and resource allocation in mobile-edge computing with inter-user task dependency," *IEEE Trans. Wireless Commun.*, vol. 19, no. 1, pp. 235–250, Jan. 2020.
- [6] S. Bi, L. Huang, and Y.-J.-A. Zhang, "Joint optimization of service caching placement and computation offloading in mobile edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 19, no. 7, pp. 4947–4963, Jul. 2020.
- [7] G. Lee, W. Saad, and M. Bennis, "An online optimization framework for distributed fog network formation with minimal latency," *IEEE Trans. Wireless Commun.*, vol. 18, no. 4, pp. 2244–2258, Apr. 2019.
- [8] S. Bi and Y. J. Zhang, "Computation rate maximization for wireless powered mobile-edge computing with binary computation offloading," *IEEE Trans. Wireless Commun.*, vol. 17, no. 6, pp. 4177–4190, Jun. 2018.
- [9] T. Q. Dinh, J. Tang, Q. D. La, and T. Q. S. Quek, "Offloading in mobile edge computing: Task allocation and computational frequency scaling," *IEEE Trans. Commun.*, vol. 65, no. 8, pp. 3571–3584, Aug. 2017.
- [10] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [11] Y. Liu, H. Yu, S. Xie, and Y. Zhang, "Deep reinforcement learning for offloading and resource allocation in vehicle edge computing and networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 11, pp. 11158–11168, Nov. 2019.
- [12] M. Min, L. Xiao, Y. Chen, P. Cheng, D. Wu, and W. Zhuang, "Learning-based computation offloading for IoT devices with energy harvesting," *IEEE Trans. Veh. Technol.*, vol. 68, no. 2, pp. 1930–1941, Feb. 2019.
- [13] J. Li, H. Gao, T. Lv, and Y. Lu, "Deep reinforcement learning based computation offloading and resource allocation for MEC," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Apr. 2018, pp. 1–6.
- [14] X. Chen, H. Zhang, C. Wu, S. Mao, Y. Ji, and M. Bennis, "Optimized computation offloading performance in virtual edge computing systems via deep reinforcement learning," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4005–4018, Jun. 2019.
- [15] M. Tang and V. W. S. Wong, "Deep reinforcement learning for task offloading in mobile edge computing systems," 2020, *arXiv:2005.02459*. [Online]. Available: <http://arxiv.org/abs/2005.02459>
- [16] Y. Wei, F. R. Yu, M. Song, and Z. Han, "Joint optimization of caching, computing, and radio resources for fog-enabled IoT using natural actor-critic deep reinforcement learning," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 2061–2073, Apr. 2019.
- [17] J. Du, F. R. Yu, G. Lu, J. Wang, J. Jiang, and X. Chu, "MEC-assisted immersive VR video streaming over terahertz wireless networks: A deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9517–9529, Oct. 2020.
- [18] L. Huang, S. Bi, and Y.-J.-A. Zhang, "Deep reinforcement learning for online computation offloading in wireless powered mobile-edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 19, no. 11, pp. 2581–2593, Nov. 2020.
- [19] L. Xiao, X. Lu, T. Xu, X. Wan, W. Ji, and Y. Zhang, "Reinforcement learning-based mobile offloading for edge computing against jamming and interference," *IEEE Trans. Commun.*, vol. 68, no. 10, pp. 6114–6126, Oct. 2020.
- [20] J. Zhang, J. Du, Y. Shen, and J. Wang, "Dynamic computation offloading with energy harvesting devices: A hybrid-decision-based deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9303–9317, Oct. 2020.
- [21] M. J. Neely, "Stochastic network optimization with application to communication and queueing systems," *Synth. Lect. Commun. Netw.*, vol. 3, no. 1, pp. 1–211, Jan. 2010.
- [22] Y. Mao, J. Zhang, and K. B. Letaief, "Dynamic computation offloading for mobile-edge computing with energy harvesting devices," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 12, pp. 3590–3605, Dec. 2016.
- [23] Y. Sun, S. Zhou, and J. Xu, "EMM: Energy-aware mobility management for mobile edge computing in ultra dense networks," *IEEE J. Sel. Areas Commun.*, vol. 35, no. 11, pp. 2637–2646, Nov. 2017.
- [24] Y. Mao, J. Zhang, S. H. Song, and K. B. Letaief, "Stochastic joint radio and computational resource management for multi-user mobile-edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 16, no. 9, pp. 5994–6009, Sep. 2017.
- [25] J. Du, F. R. Yu, X. Chu, J. Feng, and G. Lu, "Computation offloading and resource allocation in vehicular networks based on dual-side cost minimization," *IEEE Trans. Veh. Technol.*, vol. 68, no. 2, pp. 1079–1092, Feb. 2019.
- [26] C.-F. Liu, M. Bennis, M. Debbah, and H. V. Poor, "Dynamic task offloading and resource allocation for ultra-reliable low-latency edge computing," *IEEE Trans. Commun.*, vol. 67, no. 6, pp. 4132–4150, Jun. 2019.
- [27] C. Yang, J. Li, W. Ejaz, A. Anpalagan, and M. Guizani, "Utility function design for strategic radio resource management games: An overview, taxonomy, and research challenges," *Trans. Emerg. Telecommun. Technol.*, vol. 28, no. 5, p. e3128, May 2017.
- [28] L. Georgiadis, M. J. Neely, and L. Tassiulas, "Resource allocation and cross-layer control in wireless networks," *Found. Trends Netw.*, vol. 1, no. 1, pp. 1–144, 2006.
- [29] C. H. Papadimitriou and K. Steiglitz, *Combinatorial Optimization: Algorithms and Complexity*. New York, NY, USA: Dover, 1998.
- [30] S. Marsland, *Machine Learning: An Algorithmic Perspective*. Boca Raton, FL, USA: CRC Press, 2015.
- [31] J. Yan, S. Bi, and Y. J. A. Zhang, "Offloading and resource allocation with general task graph in mobile edge computing: A deep reinforcement learning approach," *IEEE Trans. Wireless Commun.*, vol. 19, no. 8, pp. 5404–5419, Aug. 2020.
- [32] K. M. Anstreicher, "Linear programming in $O([N^3/\ln n]L)$ operations," *SIAM J. Optim.*, vol. 9, no. 4, pp. 803–812, 1999.
- [33] T. P. Lillicrap *et al.*, "Continuous control with deep reinforcement learning," in *Proc. ICLR*, 2016, pp. 1–4.



Suzhi Bi (Senior Member, IEEE) received the B.Eng. degree in communications engineering from Zhejiang University in 2009, and the Ph.D. degree in information engineering from The Chinese University of Hong Kong in 2013. From 2013 to 2015, he was a Post-Doctoral Research Fellow with the Department of Electrical and Computer Engineering, National University of Singapore. Since 2015, he has been with the College of Electronics and Information Engineering, Shenzhen University, China, where he is currently an Associate Professor. His research interests include optimizations in wireless information and power transfer, mobile computing, and smart power grid communications. He received the IEEE ComSoc Asia-Pacific Outstanding Young Researcher Award in 2019 and two times Shenzhen University Outstanding Young Faculty award. He was a co-recipient of the IEEE SmartGridComm 2013 Best Paper Award. He is an Associate Editor of IEEE WIRELESS COMMUNICATIONS LETTERS.



Liang Huang (Member, IEEE) received the B.Eng. degree in communications engineering from Zhejiang University in 2009 and the Ph.D. degree in information engineering from The Chinese University of Hong Kong in 2013. He is currently an Associate Professor with the College of Computer Science and Technology, Zhejiang University of Technology, China. His research interests include queueing and scheduling in communication systems and networks.



Hui Wang received the B.S., M.S., and Ph.D. degrees in electrical engineering, and electronic and information engineering from Xi'an Jiaotong University, Xi'an, China, in 1990, 1993, and 1996, respectively. He is currently a Professor with the Shenzhen Institute of Information Technology, Shenzhen, China. His research interests include wireless communication, signal processing, and distributed computing systems, in which he has authored or coauthored over 60 journal articles, conference papers, and book chapters.



Ying-Jun Angela Zhang (Fellow, IEEE) is currently a Professor with the Department of Information Engineering, The Chinese University of Hong Kong. Her research interests include optimization and learning in wireless communication systems and smart power grids. She is a member of IEEE ComSoc Fellow Evaluation Committee. She was a co-recipient of the 2011 Marconi Prize Paper Award in Wireless Communications, the 2013 SmartGrid-Comm Best Paper Award, and the 2014 IEEE ComSoc Asia Pacific Board Outstanding Paper Award. She received the 2006 Hong Kong Young Scientist Award as the only winner from engineering science. She has served as the Chair of the Executive Editorial Committee of IEEE Transactions on Wireless Communications and a Founding Chair of IEEE ComSoc Smart Grid Communications Technical Committee. She is also an Associate Editor-in-Chief of IEEE OPEN JOURNALS OF THE COMMUNICATION SOCIETY. She is a Distinguished Lecturer of IEEE ComSoc.